

InfiniFlow: Decoupling Virtual Channel Scalability from Buffer Requirements in Lossless Datacenter Networks

Zerui Tian*
Fudan University

Sen Liu*
Fudan University

Minkun Xue
Fudan University

Hao Shangguan
Fudan University

Ruyi Yao
Fudan University

Hao Mei
Fudan University

Deli Huang
Fudan University

Songchen Xue
Fudan University

Yang Xu[†]
Fudan University

Abstract

Modern datacenters increasingly rely on Remote Direct Memory Access (RDMA) for high throughput and low latency. To ensure lossless transmission, RDMA fabrics employ hop-by-hop flow control at the granularity of Virtual Channels (VCs). However, multiplexing multiple flows within the same VC couples their backpressure behavior, leading to severe congestion spreading, degrading throughput and tail latency significantly. Existing methods try to improve isolation by increasing the number of VCs but require per-VC buffer reservation, causing buffer requirements to scale linearly with the number of VCs and fundamentally limiting VC scalability.

We present InfiniFlow, a credit-based hop-by-hop flow control method that supports massive VCs with a limited buffer budget via per-port buffer sharing. InfiniFlow introduces a paradigm shift in buffer management: Upstream Allocates Buffer for Downstream (UABD), where, for each link, the upstream scheduler maintains a per-port shared credit pool and dynamically allocates credits to VCs. Building on UABD, InfiniFlow further incorporates a Buffer Usage Control Protocol that enforces per-VC dynamic credit thresholds using real-time backlog feedback to bound credit consumption and prevent buffer monopolization.

With InfiniFlow, the required buffer size per port becomes independent of the number of VCs and is instead bounded by a small budget proportional to per-hop Bandwidth-Delay Product (BDP) (approximately $3 \times \text{BDP}$). We implement InfiniFlow on a Xilinx Alveo U280 FPGA and demonstrate support for over 16k VCs using only 401 KB of on-chip memory per port while sustaining 100 Gbps line-rate transmission, achieving $512\times$ and $1024\times$ higher VC scalability than CBFC and PFC, respectively. Extensive NS-3 simulations show that InfiniFlow achieves near-optimal flow completion times under realistic workloads without relying on slow end-to-end rate adaptation, outperforming prior hop-by-hop flow control methods and state-of-the-art congestion control schemes.

*These authors contributed equally to this work.

[†]Yang Xu is the corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGCOMM '26, Denver, CO, USA*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2467-1/26/08

<https://doi.org/10.1145/3789240.3829135>

CCS Concepts

• Networks → Network protocol design.

Keywords

Datacenter Networks, Lossless Networks, Flow Control

ACM Reference Format:

Zerui Tian, Sen Liu, Minkun Xue, Hao Shangguan, Ruyi Yao, Hao Mei, Deli Huang, Songchen Xue, and Yang Xu. 2026. InfiniFlow: Decoupling Virtual Channel Scalability from Buffer Requirements in Lossless Datacenter Networks. In *ACM SIGCOMM 2026 Conference (SIGCOMM '26)*, August 17–21, 2026, Denver, CO, USA. ACM, New York, NY, USA, 18 pages. <https://doi.org/10.1145/3789240.3829135>

1 Introduction

The rapid growth of large-scale Artificial Intelligence [26, 40] and High-Performance Computing workloads [22] has pushed Data Center Networks (DCNs) to deliver ever-increasing bandwidth with ultra-low latency. Remote Direct Memory Access (RDMA) [48] has thus become a cornerstone of modern DCNs. Because RDMA is highly sensitive to packet loss, production networks enforce lossless transmission through hop-by-hop flow control mechanisms such as Priority-based Flow Control (PFC) [17] or Credit-based Flow Control (CBFC) [41].

These mechanisms operate at the granularity of Virtual Channels (VCs)¹. On each link, upstream and downstream ports coordinate via backpressure independently per VC. However, commodity switches support only a limited number of VCs (e.g., 8 for PFC and 16 for CBFC [17, 41]). As the number of flows increases, multiple flows must share the same VC, inevitably leading to frequent VC collisions. When congestion throttles a VC, Head-of-Line Blocking (HoLB) [29, 31, 47] propagates across all flows sharing that VC, degrading throughput and inflating tail latency.

End-to-end Congestion Control (CC) attempts to mitigate HoLB by adapting source rates. However, under increasingly bursty and large-scale workloads, CC often converges slowly or inaccurately, leaving significant performance loss [16]. In contrast, per-flow flow control (where each flow is isolated in its own VC) can eliminate HoLB and achieve near-optimal rate convergence. Unfortunately, scaling VCs is fundamentally constrained by buffer requirements.

¹Throughout this paper, we use “VC” to denote a hardware-enforced logical isolation unit with independent flow control state. It may map to a PFC priority queue in Ethernet, or a virtual lane in InfiniBand CBFC.

Existing hop-by-hop flow control mechanisms reserve buffer space² per VC to guarantee line-rate lossless transmission when the VC is active³ alone. Specifically, PFC requires at least $2\times$ per-hop Bandwidth-Delay Product (BDP) [46], while CBFC requires $1\times$ per-hop BDP [45]. As a result, total buffer requirements grow linearly with the number of VCs, rendering massive per-flow VCs impractical in shallow-buffer switches [5, 6, 9, 23].

In this paper, we revisit a fundamental question: *Must buffer requirements scale with the number of VCs?* We observe that VCs multiplex over a single physical link whose aggregate transmission rate is bounded. For a fixed link rate and per-hop Round Trip Time (RTT), the buffer required to sustain lossless line-rate transmission is fundamentally bounded per port, not per VC. The conventional linear scaling arises from per-VC reservation rather than inherent necessity.

Guided by this insight, we propose a paradigm shift in buffer management: Upstream Allocates Buffer for Downstream (UABD). Instead of relying on local buffer allocation and synchronization to upstream, UABD enables the upstream port (where packets are scheduled and transmitted) to proactively manage buffer allocation on behalf of the downstream port. This eliminates activation time and synchronization overhead, enabling efficient per-port buffer sharing across VCs.

Building on UABD, we design InfiniFlow⁴, a credit-based hop-by-hop flow control mechanism that supports massive VCs under a small, constant per-port buffer budget. InfiniFlow maintains a shared credit pool at the upstream port representing available space in the downstream buffer. To prevent *Inter-VC Interference* introduced by buffer sharing, InfiniFlow introduces a Buffer Usage Control Protocol (BUCP) that dynamically caps per-VC credit consumption using real-time backlog feedback, ensuring that throttled VCs cannot monopolize shared buffer space. As a result, buffer occupancy remains tightly controlled, often limited to only a few Maximum Transmission Units (MTUs) per throttled VC.

Our contributions are as follows:

- We identify that the buffer requirement for lossless line-rate transmission is fundamentally bounded by per-hop BDP rather than the number of VCs, challenging the conventional linear scaling assumption.
- We propose InfiniFlow, an upstream-driven, credit-based flow control mechanism that decouples VC scalability from buffer requirements via per-port buffer sharing and precise per-VC usage control.
- We implement InfiniFlow on a Xilinx Alveo U280 FPGA and demonstrate support for 16k+ VCs using only 401 KB on-chip memory per port while sustaining 100 Gbps line-rate transmission, which is compatible with the shallow-memory design of modern datacenter switches [5, 6, 9, 23].
- Extensive NS-3 simulations show that InfiniFlow achieves near-optimal Flow Completion Times (FCTs) under realistic

workloads without relying on slow end-to-end rate adaptation, outperforming both classic hop-by-hop flow control and state-of-the-art congestion control schemes.

This work does not raise any ethical issues.

2 Preliminaries

RDMA relies on backpressure to prevent buffer overflows. We briefly review the two mainstream methods: PFC and CBFC, focusing on their respective buffer size requirements to sustain lossless line-rate transmission.

2.1 Priority-based Flow Control

PFC operates on a per-VC “on-and-off” backpressure governed by ingress thresholds X_{off} and X_{on} . Backpressure is triggered when queue depth exceeds X_{off} and released when it falls below X_{on} . To sustain line-rate transmission without link idling or overflows, these thresholds must offset the per-hop BDP. Specifically, X_{on} requires at least $1\times$ BDP to prevent underflow during resume, while the region above X_{off} needs another $1\times$ BDP to absorb in-flight bytes during pause. Given $X_{off} \geq X_{on}$, **PFC requires a per-VC buffer reservation of at least $2\times$ per-hop BDP.**

2.2 Credit-Based Flow Control

CBFC employs a credit mechanism. For each link, CBFC partitions the downstream buffer into dedicated per-VC regions and maintains a separate credit for each VC at the upstream port to represent per-VC available buffer space. At the upstream port, transmission on a VC is authorized only if its credit covers its head packet size. This credit mechanism enables CBFC to avoid buffer overflow regardless of the VC buffer size [25, 27]. However, the VC buffer size B impacts the maximum transmission rate R_m of this VC: given a per-hop RTT τ , R_m is bounded by B/τ [25, 27]. Therefore, for any time t , the transmission rate $r(t)$ of this VC must satisfy:

$$r(t) \leq R_m \leq \frac{B}{\tau}. \quad (1)$$

To ensure each VC reaches line rate when active alone, its buffer must satisfy $B \geq C \cdot \tau$, where C is the line rate. Thus, **CBFC requires at least $1\times$ per-hop BDP for each VC.**

In the above analysis, we consider only the packet buffer requirement of PFC and CBFC, and ignore their VC metadata bank overhead, since they support only a small number of VCs and thus this overhead is negligible compared with their packet buffer occupation.

3 Motivation

The backpressure of PFC and CBFC inevitably triggers congestion spreading [19, 39, 48]. As illustrated in Figure 1, when an egress port becomes congested, buffer occupancy rises, triggering backpressure that propagates recursively to form a cascading chain of buffer exhaustion. Any victim flow sharing a VC with these congested flows suffers from HoLB, resulting in severe throughput degradation and latency spikes.

Existing works mainly focus on mitigating HoLB via end-to-end CC. From Explicit Congestion Notification (ECN) [4, 13, 42, 48] to In-Network Telemetry (INT) [2, 7, 30, 43], CC designers have

²Beyond buffer space for packets (i.e., packet buffer), per-VC metadata also consumes on-chip memory and grows linearly with the number of VCs, but it remains negligible compared to packet buffers.

³We define a VC as active when it has packets waiting for transmission.

⁴InfiniFlow’s source code is available at github.com/zerui-tian/infiniflow.

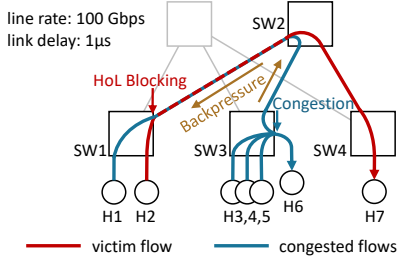


Figure 1: Congestion Spreading

resorted to increasingly granular feedback to accelerate convergence, striving to approach optimal transmission. However, this technical roadmap escalates overhead and deployment complexity, while yielding diminishing returns. This is because CCs are physically bound by the end-to-end RTT feedback lag, which limits the convergence speed and accuracy, often leading to rate oscillations and preventing sub-RTT convergence under bursty traffic [16].

3.1 The Power of Per-Flow Flow Control

In contrast, hop-by-hop flow control offers a significantly shorter control loop and achieves more precise rate adjustment via backpressure. Ideally, provided sufficient VCs to ensure per-flow isolation, standalone per-flow flow control can achieve superior performance compared to CC [16].

We demonstrate the superiority of per-flow flow control with a micro-benchmark using the small leaf-spine topology shown in Figure 1. We initiate a victim flow with data volume of 45 MB ($H2 \rightarrow H7$) at $t = 0$ ms, followed by four congested flows with data volume of 5 MB (targeting $H6$) at $t = 1$ ms. All links have 100 Gbps bandwidth and $1 \mu s$ delay. The four congested flows share a single link on the last hop, so their throughput is capped at 25 Gbps each. Therefore, ideally, the victim flow can reach up to 75 Gbps.

We analyze three scenarios: (1) VC Collision (Figure 2a): When the victim and congested flows are mapped to the same VC, HoLB prevents the victim flow from reaching its maximum 75 Gbps rate when relying solely on flow control. (2) CC Mitigation (Figure 2b): In the VC collision scenario, further enabling DCQCN [48] or HPCC [30] alleviates HoLB, improving victim throughput. However, as shown in Figure 2b, the rate adjustments are slow and imprecise. The victim flow struggles to stabilize at 75 Gbps. (3) Per-Flow Isolation (Figure 2c): When the victim flow is mapped to a separate VC from the congested flows, standalone flow control achieves the 75 Gbps throughput by applying individual backpressure to each flow, without slow rate probing.

Compared to CC, per-flow flow control offers more rapid and precise rate adjustments without requiring complex control logic. In Appendix A, we formally prove that a network applying per-flow CBFC or PFC (where each flow is assigned an independent VC) with sufficient buffer for each VC, coupled with equal-weight Deficit Round Robin (DRR) schedulers, achieves global max-min fairness in steady state.

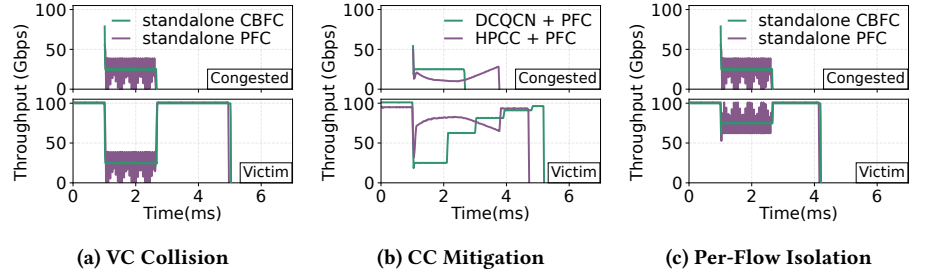


Figure 2: Congested and Victim Flow Rates in the Micro-benchmark

3.2 The Barrier to Per-Flow Flow Control

Despite its theoretical elegance, implementing per-flow flow control in practice remains a formidable challenge on commodity hardware due to the **linear buffer space scaling bottleneck**: for each VC, CBFC and PFC require a buffer reservation of at least $1\times$ and $2\times$ per-hop BDPs, respectively. In a 32-port CBFC switch with 100 Gbps links and $1 \mu s$ link delay, supporting 1000 VCs per port would require at least 800 MB of buffer, far exceeding the tens of MBs of shared SRAM available in commodity switches [5, 6, 9, 23].

Given a fixed buffer space, there is a core tension between control granularity and lossless line-rate transmission: increasing the number of VCs improves flow isolation, but reduces the buffer available to each VC. We conduct NS-3 simulations on a 1024-host leaf-spine topology with a 40% Hadoop workload, 100 Gbps links, and $1 \mu s$ link delay, measuring average FCT slowdown across switch buffer budgets and the number of VCs for CBFC/PFC and CC schemes. Additionally, we compute an ideal baseline, where instantaneous flow rates achieve global max-min fairness at every moment. Our results, depicted in Figure 3, yield two observations:

Observation 1: Increasing the Number of VCs under Insufficient Per-VC Buffer Space Can Be Counterproductive. In Figure 3a and 3b, under a fixed buffer budget, the performance of CBFC and PFC follows a U-shaped curve relative to the number of VCs, indicating the existence of a performance sweet spot. The initial downward trend reflects the benefits of flow isolation. However, once the number of VCs exceeds the sweet spot, performance deteriorates because statically dividing the buffer among too many VCs leaves insufficient space for each to sustain line-rate lossless transmission. Specifically, CBFC is throttled by credit starvation, while PFC suffers from frequent packet drops and retransmissions due to buffer overflows. Thus, while increasing VCs is a potent defense against HoLB, it becomes counterproductive when constrained by an insufficient buffer budget.

Observation 2: Per-Flow Flow Control Requires an Impractically Large Buffer to Reach Its Full Potential. In Figure 3a and 3b, as the buffer budget increases, the sweet spot shifts toward more VCs, yielding superior performance. With an impractically large buffer (more than 500 MB), the performance of standalone flow control surpasses that of CCs, closely approximating the ideal baseline. This demonstrates that standalone flow control reaches its full potential only when (1) the number of VCs is sufficient to ensure fine-grained flow isolation, and (2) the buffer budget is large enough to sustain line-rate lossless transmission for each VC. Unfortunately, meeting

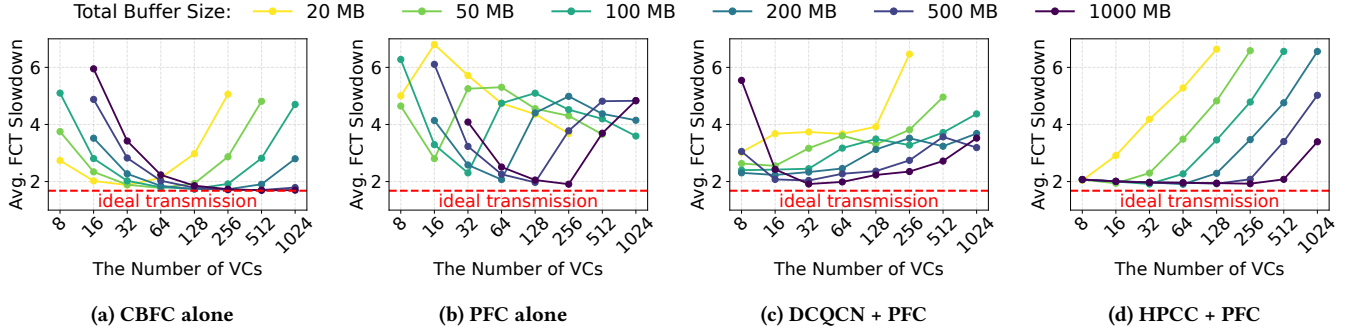


Figure 3: Average FCT Slowdown under Hadoop Workload with 40% Intensity in a 1024-host Leaf-spine Topology.

both conditions is blocked by the buffer bottleneck, making per-flow flow control impractical under the traditional per-VC buffer reservation architecture.

To avoid the buffer bottleneck constraining VC scalability, some solutions, such as BFC [16], approximate per-flow flow control by dynamically mapping flows to a limited pool of VCs. However, this approach faces significant limitations: (1) it incurs substantial complexity and per-flow state overhead; (2) under heavy loads, performance degrades as the scarcity of VCs prevents sufficient flow isolation [16, 29].

Therefore, to achieve true per-flow flow control without performance compromises, we must challenge the underlying architectural constraint. In this paper, we ask: *Is it possible to decouple VC scalability from the linear growth of buffer space requirements, thereby supporting a massive number of VCs under a limited buffer budget?* To identify the opportunities for such decoupling, we first analyze the inefficiency of buffer management used by current flow control methods.

3.3 Inefficiency of Static Buffer Allocation

Buffer management in current flow control methods adopts static allocation, reserving a buffer for each VC which ensures that a VC can sustain line-rate lossless transmission when it is active alone. However, this approach is inefficient in practice. First, VCs are rarely all active simultaneously [16]; thus, per-VC reservation causes valuable buffer space to be stranded in inactive VCs rather than serving active ones. Second, and also more importantly, even when multiple VCs are concurrently active, they still multiplex over the same physical link, so their aggregate transmission rate is bounded by the line rate. Furthermore, because the buffer requirement of each VC scales with its sending rate, the aggregate buffer space required by each port is ultimately bounded by the line rate rather than the number of VCs.

Taking CBFC as a case, consider a link with N VCs and assume that VC_i transmits at rate $r_i(t)$ at time t . From Equation 1 in § 2, sustaining $r_i(t)$ requires a minimum buffer of $r_i(t) \cdot \tau$, where τ is the per-hop RTT. Under the physical bandwidth constraint $\sum_{i=1}^N r_i(t) \leq C$, at any time t , the minimum aggregate buffer required across all VCs, $B_{agg}(t)$, satisfies:

$$B_{agg}(t) = \sum_{i=1}^N r_i(t) \cdot \tau \leq C \cdot \tau. \quad (2)$$

This implies that regardless of how rates are allocated across VCs, the theoretical aggregate buffer required by each port is bounded by $1 \times$ per-hop BDP (in practice, we need around $3 \times$ per-hop BDPs per port for optimal performance, see § 6.3), leading to our **key insight**: *the linear growth in required buffer space stems from rigid per-VC reservations rather than actual demand. If VCs on a port can share a common buffer, their aggregate buffer space required to sustain lossless line-rate transmission becomes bounded by a small constant on the order of per-hop BDPs, regardless of the number of VCs.*

4 InfiniFlow Design

Based on our key insight, we propose InfiniFlow, a credit-based flow control method that supports massive VCs under a limited buffer budget via per-port buffer sharing. InfiniFlow has three key properties:

- (I) **Lossless transmission**: With the credit-based design, InfiniFlow guarantees that buffers never overflow.
- (II) **Immediate buffer allocation**: InfiniFlow can allocate downstream buffer to different VCs immediately at the upstream node without transmission stalls.
- (III) **VC demand awareness**: InfiniFlow’s buffer allocation is demand-aware, constraining each VC’s buffer usage to its transmission demand, thereby preventing buffer monopolization while maximizing throughput.

Next, we present an overview of InfiniFlow.

4.1 InfiniFlow Overview

InfiniFlow builds upon a buffer-sharing paradigm shift: *for each link, the upstream port performs buffer space allocation on behalf of the downstream port, namely, UABD.*

Rationale of UABD. In traditional hop-by-hop flow control, each switch’s buffer allocation is determined by its local buffer manager, which then reports the per-VC available buffer size to the upstream switch, as shown in Figure 4a. The upstream switch regulates per-VC transmissions according to the allocation decision. With per-port buffer sharing, however, the buffer manager must additionally track which VCs are currently active at the upstream switch, allocate buffer to active VCs locally, and synchronize the allocation back upstream. This synchronization introduces $1 \times$ per-hop RTT latency when activating VCs or resizing buffer allocations,

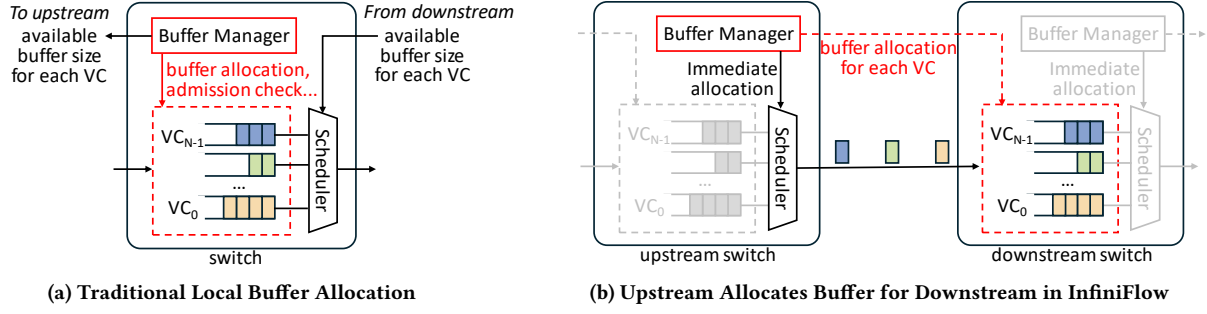


Figure 4: To enable immediate buffer allocation, InfiniFlow introduces a paradigm shift in which upstream nodes allocate downstream buffer.

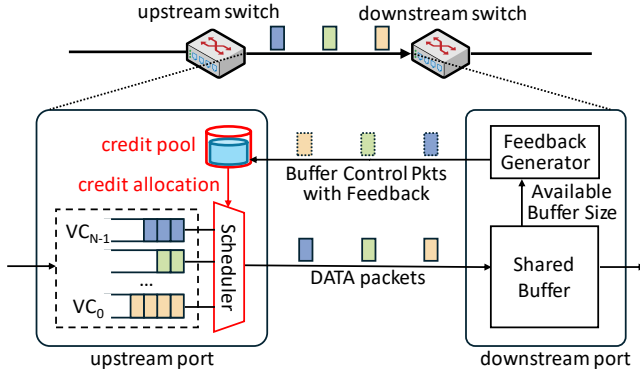
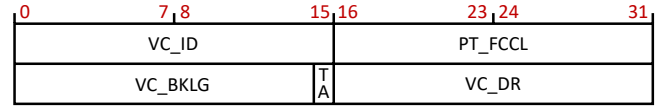


Figure 5: InfiniFlow Overview

leading to transmission stalls. To eliminate these stalls, buffer allocation must be integrated with VC state maintenance in the same entity. The insight is that the upstream port possesses complete visibility into VC states earlier than the downstream port, as it is the egress point where packets depart. More significantly, upstream ports can proactively schedule packets across VCs, directly performing buffer space allocation on behalf of the downstream port. This is the philosophy of UABD, as shown in Figure 4b.

Overview of InfiniFlow. To deploy UABD, as shown in Figure 5, for each link InfiniFlow maintains a per-port credit pool at the upstream port that represents the available shared buffer in the downstream port. The upstream scheduler allocates credits from this pool to VCs, effectively enabling the upstream port to perform buffer space allocation on behalf of the downstream port and thus achieving (I) **lossless transmission** and (II) **immediate buffer allocation**. § 4.2 describes how we maintain the per-port credit pools.

Credit Allocation Challenge. Under this architecture, credit allocation is crucial because, without per-VC buffer reservations in existing flow control methods, throttled VCs can abuse excess buffer space, starving innocent VCs [25], which we denote as *Inter-VC Interference*. Dynamic Threshold (DT) [11], a widely adopted technique in shared-memory switches, may seem like a possible solution. DT can evenly allocate credits among active VCs, preventing throttled VCs from monopolizing the shared buffer, but it is not VC demand-aware and thus cannot fully eliminate *Inter-VC Interference*. We demonstrate this with a micro-benchmark simulation in Appendix C. The limitation of DT underscores the need



VC_ID: a 16-bit field identifies which VC the packet belongs to
PT_FCCL: a 16-bit field for buffer availability advertisement (credit feedback)
VC_BKLG: a 15-bit field for real-time VC backlog feedback
TA_FLAG: a 1-bit indicator for rate oscillation avoidance
VC_DR: a 16-bit field conveys the downstream cumulative drained bytes

Figure 6: The Format of Buffer Control Packets

for a (III) **VC-demand-aware** credit allocation protocol, ensuring that individual transmission demands are fully met while strictly preventing buffer monopolization, as detailed in § 4.3 and § 4.4.

Downstream Feedback Format. The enforcement of credit pooling and allocation relies on downstream feedback as shown in Figure 5. Each time a packet is retrieved from the downstream buffer, the downstream port encapsulates the required feedback into a Buffer Control Packet (BCP) and sends it upstream. The BCP format is shown in Figure 6. Next, we describe in detail how credit pooling and allocation operate.

4.2 Per-Port Credit Pool Maintenance

To enable UABD, InfiniFlow establishes a `credit_pool` for each link at the upstream port to represent the total availability of the shared downstream buffer, thereby empowering the upstream scheduler to make real-time credit allocation decisions without waiting for remote grants. An active VC is eligible for transmission only if the current `credit_pool` can cover the size of its head packet to avoid buffer overflow; this forms InfiniFlow's **Per-Port Scheduling Constraint**:

$$Head_Pkt_Size \leq credit_pool. \quad (3)$$

The computation of `credit_pool` is patterned after CBFC's counter-based mechanism, but shifts the counter semantics from per-VC to per-port, as shown in Figure 7. The mechanism requires the Per-Port TX Bytes Counter (denoted as `PT_TX`) and Per-Port RX Bytes Counter (denoted as `PT_RX`), which track the cumulative transmitted and received bytes for the upstream and downstream ports, respectively. Each time a data packet is retrieved from the downstream buffer, the downstream port generates a BCP and computes `PT_FCCL` as the sum of `PT_RX` and the available shared buffer size, and then piggybacks it in the BCP back to the upstream port. `PT_FCCL` serves a similar role to the Flow Control Credit Limit (FCCL) field in

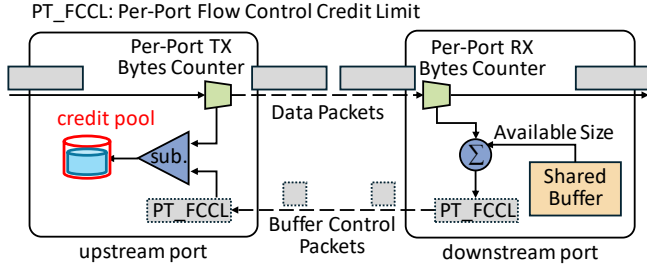


Figure 7: The Maintenance of Per-port Credit Pool

CBFC [41]. Upon receiving the BCP, the upstream port caches the latest PT_FCCL and updates $credit_pool = PT_FCCL - PT_TX$. In addition, whenever the upstream port transmits a data packet, PT_TX increases, resulting in the recomputation of $credit_pool$ using the locally cached PT_FCCL at the upstream port.

4.3 Adaptive Per-VC Buffer Usage Control

To address *Inter-VC Interference*, we propose BUCP for buffer usage control, which performs demand-aware credit allocation across VCs while preserving VC isolation. The key idea of BUCP is to *cap credit consumption for each VC at just what it needs to meet transmission demand with real-time per-VC backlog feedback*. This means VCs can sustain their maximum throughput without allowing throttled VCs to occupy excess credits and starve others.

InfiniFlow maintains an $inflight_bytes$ variable for each VC to track credit consumption, representing the amount of credit currently occupied by that VC, and uses a per-VC threshold to cap $inflight_bytes$ for each VC individually. The correct threshold should be just enough to meet the demand of the VC while minimizing the downstream VC backlog. An active VC is eligible for transmission only if its remaining quota (i.e. $threshold - inflight_bytes$) can cover its head packet size; this forms the **Per-VC Constraint**:

$$Head_Pkt_Size \leq threshold - inflight_bytes. \quad (4)$$

The key to driving the threshold toward its correct value is to determine whether a VC is currently over- or under-consuming credits. We observe that the downstream backlog provides a real-time signal of this imbalance: a near-zero backlog suggests the VC is under-credited, whereas a persistent backlog indicates credit over-allocation. Therefore, InfiniFlow enforces threshold adjustment based on the per-VC downstream backlog volume. Next, we describe how InfiniFlow computes $inflight_bytes$ and adjusts threshold.

The Computation of Per-VC Credit Consumption. The calculation of $inflight_bytes$ is shown in Figure 8, which represents the amount of credit that a VC has spent but has not yet been recycled. We observe that this notion closely resembles the TCP sending window, which represents the number of bytes transmitted but not yet acknowledged.

Inspired by the TCP sending window, InfiniFlow maintains two per-VC counters: The Per-VC TX Bytes Counter (denoted as VC_TX) at the upstream port records the cumulative bytes transmitted by the upstream port for this VC; the Per-VC Drained Bytes Counter (denoted as VC_DR) at the downstream port records the cumulative bytes drained from the downstream buffer for this VC. When a packet of a VC is dequeued from the downstream buffer and triggers

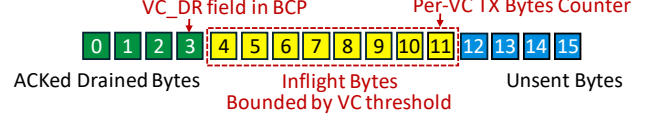


Figure 8: Per-VC Inflight Bytes at Upstream Port

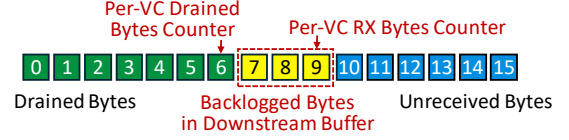


Figure 9: Per-VC Backlogged Bytes at Downstream Port

a BCP, the downstream port piggybacks the latest VC_DR in the BCP and sends it back upstream. Upon receiving the BCP, the upstream port updates $inflight_bytes = VC_TX - VC_DR$, which provides an exact measure of per-VC credit consumption, including the bytes on the link and the bytes currently buffered downstream.

Set-point Threshold Adjustment. Our threshold adjustment logic depends on the downstream backlog volume of each VC. To compute the per-VC backlog, InfiniFlow relies on the Per-VC Drained Bytes Counter VC_DR and an additional Per-VC RX Bytes Counter (denoted as VC_RX), which tracks the cumulative bytes received by the downstream port for that VC, shown in Figure 9.

When a BCP is generated, the downstream port computes the real-time backlog of the corresponding VC as $VC_RX - VC_DR$ and piggybacks it in the VC_BKLG field of this BCP. Based on the VC_BKLG field, the upstream port employs a *Set-point Adjustment* strategy to adjust the per-VC threshold rather than using slow incremental probing (e.g., AIMD-style tuning). This strategy includes two parameters, $qmin$ and $qmax$. When the upstream port observes that VC_BKLG is below $qmin$ or not smaller than $qmax$, it triggers a threshold adjustment:

```
if (VC_BKLG >= qmax):
    # congestion suppression
    threshold = inflight_bytes - VC_BKLG + qmin
if (VC_BKLG < qmin):
    # throughput elevation
    threshold += (qmin - VC_BKLG)
```

We illustrate the rationale behind this threshold adjustment algorithm with two scenarios:

Scenario A: Congestion Suppression. A backlog exceeding a safety margin ($qmax$) indicates that the VC has consumed surplus credits beyond its demand. To eliminate this “bloat” rapidly, the upstream port immediately reduces the VC’s threshold to a target value: $inflight_bytes - VC_BKLG + qmin$. The rationale for this value is that, when we use $qmin$ as the target backlog, the term $(VC_BKLG - qmin)$ represents the exact volume of surplus credits trapped in the queue. By subtracting this amount from the current consumption ($inflight_bytes$), InfiniFlow forces the upstream scheduler to pause transmission (inserting “bubbles”) until exactly that amount of data drains, effectively resetting the backlog to the target $qmin$. In addition, to prevent stagnation of VCs, the minimum threshold for each VC is set to 1 MTU.

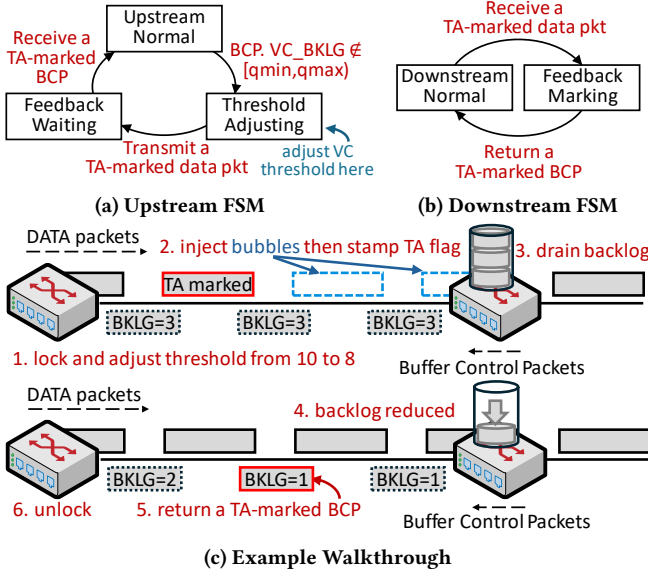


Figure 10: Locking Mechanism for Avoiding Oscillation Caused by Frequent Threshold Adjustment

Scenario B: Throughput Elevation. Conversely, if the backlog is below q_{min} , it suggests that the VC is under-credited. The protocol then increases the threshold by $(q_{min} - VC_BKLG)$, allowing the VC to ramp up and utilize spare bandwidth until the target queue occupancy is established. In the absence of downstream congestion, where the target queue occupancy cannot be established, the threshold is driven to its maximum value (the entire shared buffer size). The threshold of each VC is also initialized to the maximum value.

Notably, the above threshold adjustment does not require any *a priori* knowledge of link-specific parameters (e.g., line rate or propagation delay) and instead adapts purely based on the observed per-VC backlog feedback.

4.4 Threshold Oscillation Avoidance

To prevent oscillations due to frequent threshold adjustments, BUCP includes a locking mechanism ensuring that a new adjustment is initiated only after the effects of the previous one have taken effect. Figure 10a and 10b illustrate this interaction using Finite State Machines (FSMs) initialized at Upstream Normal and Downstream Normal. Figure 10c presents an example to illustrate how the locking mechanism works. Consider a VC with $inflight_bytes = 10$, $threshold = 10$, and parameters $q_{max} = 3$, $q_{min} = 1$. Initially, the downstream buffer contains 3 packets.

At first, the downstream port reports $VC_BKLG = 3$ to the upstream port. Detecting $VC_BKLG \geq q_{max}$, the upstream port calculates the new threshold as $10 - 3 + 1 = 8$ and transitions to Threshold Adjusting state to block further threshold updates. Then, since the new threshold is lower than the current consumption ($inflight_bytes = 10$), the scheduler pauses transmission for this VC. This effectively injects two “bubbles” (idle slots) into the pipeline. After that, the upstream port marks the *next* transmitted data packet with a TA flag (reusing ECN bits in the IP header of the data packet) and enters the Feedback Waiting state.

When the TA-marked packet reaches the downstream port, it signifies that the “bubbles” have arrived and the backlog has reduced to q_{min} . The downstream port transitions to Feedback Marking, echoes the TA flag in the next BCP, and returns to Downstream Normal. Upon receiving the echoed TA flag, the upstream port releases the lock (returning to Upstream Normal), ready for the next cycle.

Summary. InfiniFlow combines upstream credit pooling (real-time credit allocation) and backlog-driven credit allocation (demand-aware VC isolation) and makes flow control with per-port buffer sharing practical. In Appendix B, we provide a consolidated view of InfiniFlow for quick reference.

4.5 Practical Considerations

Finally, we discuss three practical considerations of InfiniFlow.

BCP Transmission Priority. Similar to PFC and CBFC, InfiniFlow carries flow-control feedback in-band over the same physical link as data traffic. To reduce buffer requirements and avoid control oscillations caused by delayed feedback, BCPs are assigned strict priority over data packets on the reverse path, ensuring that they are not blocked by data packet congestion.

Counter Overflow. Although the use of incremental byte counters provides inherent resilience against BCP loss, it introduces the challenge of counter overflow. Since InfiniFlow shares the same counter-based logic as CBFC, we adopt a similar industry-standard wrap-safe modular arithmetic approach [14]. This approach ensures that the upstream port can always correctly interpret the downstream state without requiring a complex global reset of the credit system.

Practical Buffer Size Requirement. As discussed in § 3.3, a credit-based design ideally needs only $1\times$ per-hop BDP of buffering per port to sustain line rate. In practice, however, $1\times$ per-hop BDP can cause underflow because credits are not always fully in flight: propagation delay prevents downstream decisions from taking effect instantly, and abrupt congestion may temporarily lock credits until BUCP trims the backlog. With only $1\times$ per-hop BDP, this locking can reduce circulating credits below the amount needed for line-rate transmission. To quantify InfiniFlow’s buffer requirement, § 6.3 evaluates performance versus buffer budget on a 1024-host leaf-spine topology with the Hadoop workload, showing that even at 80% load, $3\times$ per-hop BDP per link achieves near-optimal performance.

5 InfiniFlow Prototype

We implement a prototype of InfiniFlow on the Xilinx Alveo U280 FPGA platform [3]. Operating at a 100 Gbps line rate via QSFP28 optics, the prototype realizes InfiniFlow’s credit and buffer management logic. It serves as a single-hop validation setup, instantiating both upstream and downstream endpoints to emulate two ports of a physical link.

In addition to the modules in § 4, our prototype implements a *Packet Buffer* to store packets and a *Metadata Bank* to maintain all per-VC states. The bank is partitioned into two components. The *Upstream Metadata Bank* stores each VC’s flow-control context, including transmission counters, and dynamic thresholds. The *Downstream Metadata Bank* stores each VC’s queue management

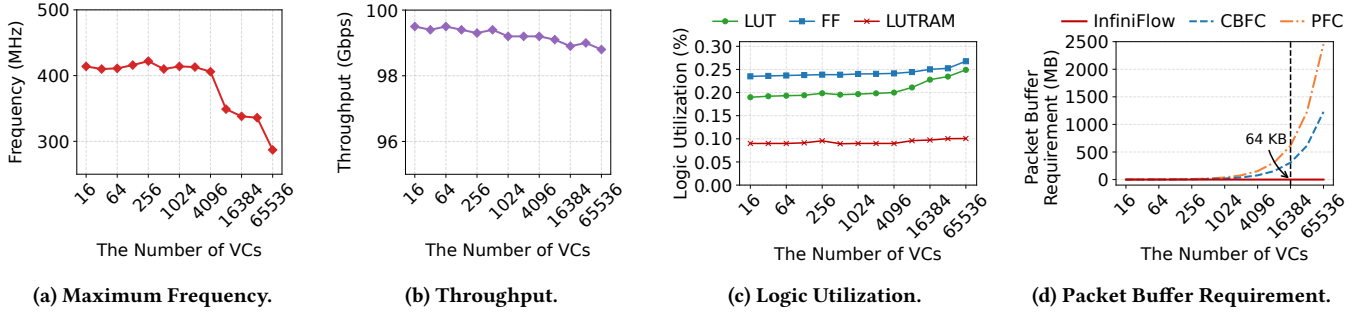


Figure 11: FPGA Prototype Evaluation on Xilinx Alveo U280 Platform

context, including linked-list head/tail pointers for chaining non-contiguous buffer segments, as well as backlog statistics used to generate feedback.

Pipelined Datapath and Computation Footprint. The prototype implements the per-packet computation logic in §4 as a pipelined hardware datapath. Because per-packet processing follows a simple unidirectional workflow with no dependencies or structural hazards, the pipeline sustains line rate while hiding metadata/buffer access latency. The computation footprint is lightweight, consisting mainly of counter/threshold updates and comparisons.

Memory Footprint and Scalability. Beyond fixed overheads introduced by the platform, the design-determining on-chip memory footprint of InfiniFlow is captured by two components: the Packet Buffer and the Metadata Bank. The Packet Buffer has an inherent cost of buffering in lossless, high-utilization operation. As discussed in §6.3, a $3\times$ per-hop BDP Packet Buffer is sufficient and independent of the number of VCs. The metadata bank scales linearly with the number of VCs and consumes about 20 B per VC (variance may arise from alignment operations and compiler optimizations of the platform). Therefore, the total on-chip memory requirement per InfiniFlow port is:

$$M_{\text{InfiniFlow}} \approx \underbrace{\alpha \times \text{HBDP}}_{\text{Packet Buffer } (O(1))} + \underbrace{20 \text{ B} \times N_{\text{VC}}}_{\text{Metadata } (O(N_{\text{VC}}))}, \quad (5)$$

where α is a tunable parameter, with a typical value of 3, and HBDP is the per-hop BDP.

For comparison, CBFC incurs a substantially larger memory footprint:

$$M_{\text{CBFC}} \approx N_{\text{VC}} \times \text{HBDP} + M_{\text{state}}, \quad (6)$$

where M_{state} denotes the memory footprint of control state, which is implementation-dependent.

With 16384 VCs, InfiniFlow's Metadata Bank overhead is only ≈ 320 KB, enabling massive concurrency with modest memory. The same budget can support fewer than ≈ 17 VCs on a 100 Gbps link with $1.5 \mu\text{s}$ per-hop RTT in CBFC.

6 Evaluation

In this section, we evaluate InfiniFlow's performance using a Xilinx Alveo U280 FPGA platform and NS-3 simulations.

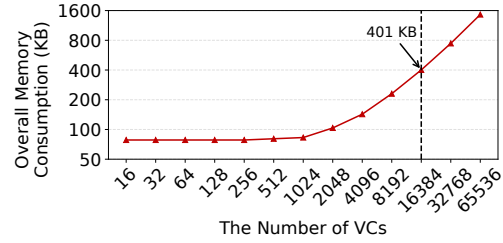


Figure 12: Overall Memory Consumption of InfiniFlow.

6.1 VC Scalability Evaluation

We use the prototype in §5 to evaluate InfiniFlow's VC scalability. Figure 11 shows InfiniFlow's performance with varying numbers of VCs in three dimensions: timing, throughput, and resource consumption. We provide 64 KB packet buffer (around $3\times$ per-hop BDP) for the evaluations.

Maximum Achievable Operating Frequency. Operating frequency is critical for sustaining line-rate processing. To explore the physical limits of InfiniFlow logic, we measure the maximum achievable operating frequency F_{max} under different numbers of VCs, scaling up to 65536 VCs. As shown in Figure 11a, the system maintains a high F_{max} (> 400 MHz) for configurations up to 4096 VCs, attributed to the extensive use of register-based pipelining. As the scale expands to 65536 VCs, F_{max} gradually decreases to approximately 290 MHz. This decline is driven by increased routing congestion and the overhead of address decoding for large memory arrays.

Line-rate Throughput Stress Test. Although the F_{max} decreases at extreme scales, timing constraints do not constitute a bottleneck for InfiniFlow. Our scheduler logic is adapted from an open-source high-performance NIC, Corundum [15]. Its highly optimized scheduling mechanism ensures that a scheduling decision is made every clock cycle. Furthermore, all state access and memory operations employ a fully pipelined design. Combined with a 512-bit data path, our prototype enables the system to provide a theoretical raw throughput of around 148 Gbps ($290 \text{ MHz} \times 512 \text{ bits}$), even for 65536 VCs. We measure the actual achieved throughput of our prototype under different numbers of VCs with up to 65536 concurrent data flows. The results in Figure 11b validate our analysis: even with 65536 VCs, the achieved throughput remains close to the 100 Gbps line-rate.

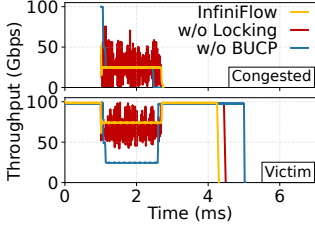
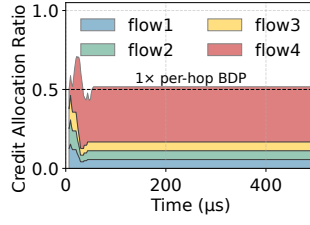
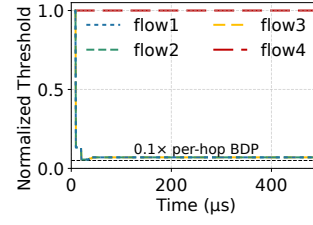


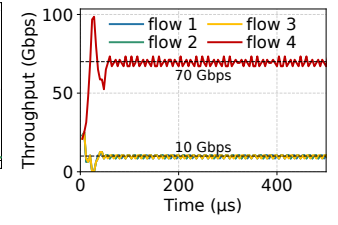
Figure 13: Ablation Study



(a) Credit Allocation Ratio



(b) Per-VC Threshold



(c) Flow Throughput

Figure 14: InfiniFlow Credit Allocation and Control Dynamics

Logic Resource Consumption. Figure 11c shows the logic resource consumption of our prototype. When the number of VCs increases, the logic resource consumption varies only slightly because we adopt a unified pipeline architecture that avoids per-VC replication of control logic: a single set of arithmetic units is time-multiplexed across VCs. This approach decouples logic complexity from VC scalability.

Memory Consumption. Memory consumption has two main components: packet buffers and VC metadata banks. For packet buffers, InfiniFlow uses a fixed 64 KB budget yet can sustain concurrent line-rate transmission with up to 65536 VCs. In contrast, as shown in Figure 11d, on a 100 Gbps link with $1.5 \mu\text{s}$ per-hop RTT, supporting tens of thousands of VCs with CBFC or PFC would require hundreds of MB of packet buffer per port, which is completely impractical.

For VC metadata banks, although each VC consumes only about 20 B of memory, the aggregate overhead becomes substantial at tens of thousands of VCs. We conduct an experiment to evaluate overall memory consumption of our prototype under different numbers of VCs. The result is shown in Figure 12. Beyond 1024 VCs, the VC metadata bank overhead becomes dominant, driving overall memory consumption to grow linearly with the number of VCs. Next, we analyze its impact on InfiniFlow's VC scalability.

VC Scalability Analysis. Consider a 64-port 100 Gbps switch with a typical maximum fiber length of 100 meters [12, 32], corresponding to a per-hop RTT of $1.5 \mu\text{s}$ (including $1 \mu\text{s}$ round-trip propagation delay and 500 ns processing delay [38]), and equipped with 40 MB on-chip memory (625 KB per port). Under this setting, InfiniFlow supports 16384 VCs using 401 KB on-chip memory per port. Since CBFC must reserve at least $1 \times$ per-hop BDP (18.75 KB) of packet buffering for each VC, a 40 MB SRAM can support only 32 VCs. For PFC, this value is halved to 16 VCs. Therefore, InfiniFlow achieves $512 \times$ and $1024 \times$ higher VC scalability than CBFC and PFC, respectively.

Prototype Fidelity Validation. In Appendix E, under exactly the same experimental setup, we measure the evolution of several key variables in InfiniFlow's control logic on both our FPGA testbed and NS-3 simulation. The results show highly consistent fluctuations in these variables across the two platforms, confirming that our FPGA prototype is a faithful implementation of the InfiniFlow design.

6.2 Micro-benchmarks

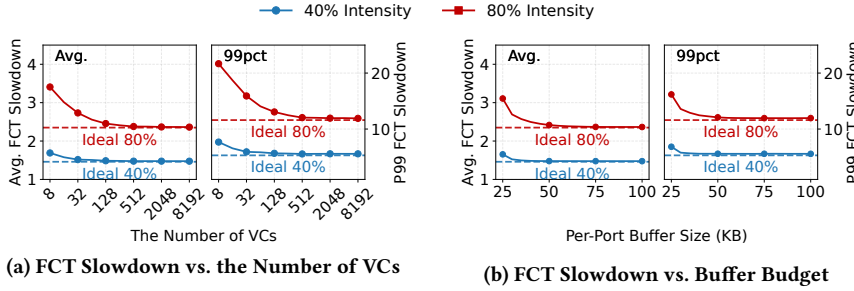
To demonstrate the micro-scale performance of InfiniFlow, we conduct a series of micro-benchmarks.

Ablation Study. InfiniFlow comprises three core mechanisms: UABD, BUCP, and the BUCP locking mechanism. Using the leaf-spine topology and traffic in Figure 1, we evaluate victim and congested flow throughput in three settings: UABD only, UABD+BUCP without locking, and full InfiniFlow. We set $q_{\min} = 1$ MTU, $q_{\max} = 3$ MTUs. Figure 13 shows the results.

As in Figure 2, the victim flow starts at 0 ms, followed by four congested flows at 1 ms. With UABD alone (w/o BUCP), buffer monopolization by congested flows caps the victim's throughput at 25 Gbps during overlap. Adding BUCP without locking (w/o Locking) raises the victim's throughput but induces severe rate oscillations. With locking enabled (InfiniFlow), oscillations are largely suppressed, approaching max-min fairness. InfiniFlow also significantly outperforms DCQCN and HPCC under the same conditions (Figure 2b).

Buffer Allocation Test. To evaluate InfiniFlow's buffer allocation efficiency, we use a dumbbell topology where four flows (Flows 1–4, each in a distinct VC) share a 100 Gbps link with $5 \mu\text{s}$ delay. Flows 1–3 are throttled to 10 Gbps by downstream links, while Flow 4 is unconstrained. The downstream buffer on the bottleneck link is sized at $2 \times$ per-hop BDP, with $q_{\min} = 1$ MTU, $q_{\max} = 3$ MTUs. Figure 14 shows stacked credit-allocation ratios (which can represent per-VC downstream buffer allocation), the threshold timeline (normalized by total shared buffer), and throughput for the four concurrent flows (VCs) under InfiniFlow.

When transmission begins, the rates of Flows 1–3 exceed 10 Gbps. Since their downstream service rate is capped at 10 Gbps, backlog accumulates in the downstream buffer. This triggers threshold adjustment, temporarily reducing their rates below 10 Gbps to reduce backlog. After several rounds, their buffer allocation converges to just enough to sustain 10 Gbps ($0.1 \times$ per-hop BDP), yielding minimal steady backlog. In contrast, Flow 4 sees no downstream congestion and maintains zero backlog, so its threshold stays at the maximum and it fully utilizes the remaining 70 Gbps. As shown in Figure 14, the aggregate buffer allocation stabilizes slightly above $1 \times$ per-hop BDP (due to the minimal backlog of Flows 1–3), validating our key insight. Overall, the result shows that InfiniFlow allocates credit in a VC-demand-aware manner, meeting each VC's demand while preventing buffer monopolization.



parameter	$q_{min} = 2$	$q_{min} = 3$	$q_{min} = 5$
(packets)	$q_{max} = 4$	$q_{max} = 6$	$q_{max} = 10$
40% Intensity	1.4811	1.4758	1.4720
80% Intensity	2.3680	2.3661	2.3660

Table 1: Sensitivity to q_{min} and q_{max} with Average FCT Slowdown

Figure 15: Sensitivity to the Number of VCs and Buffer Budget

6.3 Sensitivity Analysis

To understand how InfiniFlow’s key parameters impact performance, we evaluate the average FCT slowdown under different configurations on a 1024-host leaf-spine topology using the Hadoop workload at two intensities (40% and 80%). All links run at 100 Gbps with a delay of 1 μ s. Each switch has 64 ports; thus, allocating 1 $\times$ per-hop BDP of buffering per port requires at least 1.6 MB total buffer per switch. Unless otherwise stated, each flow is randomly mapped to a VC.

Sensitivity to the Number of VCs. We quantify how many VCs are needed in realistic settings. Figure 15a reports the average and tail FCT slowdown under a varying number of VCs with sufficient buffer budgets. Under both intensities, the gain from increasing VCs diminishes rapidly: once the number of VCs reaches 1024, further increasing it to 2048 yields little improvement in both average and tail FCTs. Therefore, we use 1024 VCs as a sufficient operating point, even under heavy load (80% intensity).

Sensitivity to Per-Port Buffer Budget. § 4.5 notes that with only 1 $\times$ per-hop BDP buffer per port, InfiniFlow cannot reach peak performance due to transient credit locking. With a sufficiently large number of VCs, we quantify the per-port buffer needed for InfiniFlow to approach ideal transmission. Figure 15b plots average and tail FCT slowdown as we vary the per-port buffer budget with the number of VCs fixed at 2048. InfiniFlow becomes near-ideal once per-port buffering reaches 3 $\times$ per-hop BDP (75 KB), indicating that 3 $\times$ per-hop BDP per port suffices even under heavy load (80% intensity).

Sensitivity to q_{min} and q_{max} . BUCP includes two parameters: q_{min} , the target backlog for per-VC threshold adjustment, and q_{max} , the maximum tolerable backlog. In Appendix D.1, we use a micro-benchmark to show that both overly small and overly large parameter settings may degrade performance. To evaluate InfiniFlow’s sensitivity to these parameters, we measure its performance under three parameter settings (in units of a 1500-byte packet), with 2048 VCs and 3 $\times$ per-hop BDP per port. Table 1 reports the average FCT slowdown. Within this range, increasing q_{min} and q_{max} yields small but consistent improvements at both intensities, as moderately larger values stabilize the control loop and reduce oscillations. Importantly, the results also show that InfiniFlow does not require a large standing queue for rate stability: maintaining only a few MTUs of steady-state backlog is sufficient.

In Appendix D.2, we further conduct a broader parameter sweep under different network conditions, including traffic intensity, application workload, and link delay. Although the exact optimal setting

varies across network conditions, InfiniFlow is not sensitive to q_{min} and q_{max} : across all evaluated cases, the maximum degradation in average FCT slowdown is only 1.83%, demonstrating its robustness to parameter choices.

6.4 Large-Scale Simulations

To evaluate the macro-scale performance of InfiniFlow in large-scale network environments, we conduct a series of macro-benchmarks, beginning with the experimental setup.

Topology and Workloads. We evaluate InfiniFlow on a large-scale leaf-spine topology with 8 spine and 32 leaf switches. Each leaf connects to 32 hosts (1024 total) at 4:1 oversubscription. Each 64-port switch has a 40 MB shared buffer, and all links run at 100 Gbps with 1 μ s delay. We use two realistic workloads from a Google WebSearch application and a Meta Hadoop cluster. We define load intensity as the average host injection rate normalized to line rate.

Comparisons. We compare InfiniFlow against two mainstream flow control methods, three representative CC protocols, and our closest competitor BFC [16]:

Flow Controls: We configure PFC and CBFC with 8 and 16 VCs. For PFC, Xon/Xoff are set to 1/3 and 2/3 of the per-port buffer. Our per-port buffer allocation is sufficient for line-rate lossless transmission for both methods.

Congestion Controls: We evaluate three CCs, all running over PFC to prevent buffer overflow: DCQCN (ECN-based), HPCC (INT-based), and ExpressPass [10] (receiver-driven). We use the recommended settings from their papers.

Our Closest Competitor BFC: BFC pioneered per-flow flow control and thus is a key precursor to our work, but it does not guarantee losslessness [16]. With a 40 MB switch buffer, using 128 VCs leaves an overly small buffer space per VC and causes severe packet loss. Therefore, in this experiment, we configure BFC with 32 VCs. Lost packets are retransmitted.

We implemented NS-3 simulation models based on [21].

Performance Metrics. We use two metrics: (1) FCT slowdown: The ratio of the observed FCT to the ideal FCT for a flow of the same size. We report the average, P95, and P99 FCT slowdowns; (2) overall buffer occupancy at the switches.

Parameter Setting. We configure InfiniFlow with 1024 VCs, where each flow is randomly mapped to a VC. The q_{min} and q_{max} are set to 5 and 10 MTUs, respectively.

Performance Analysis. Figure 16 and Figure 17 report the results. We analyze them along three dimensions: short-flow tail and long-flow average FCT, and buffer occupancy.

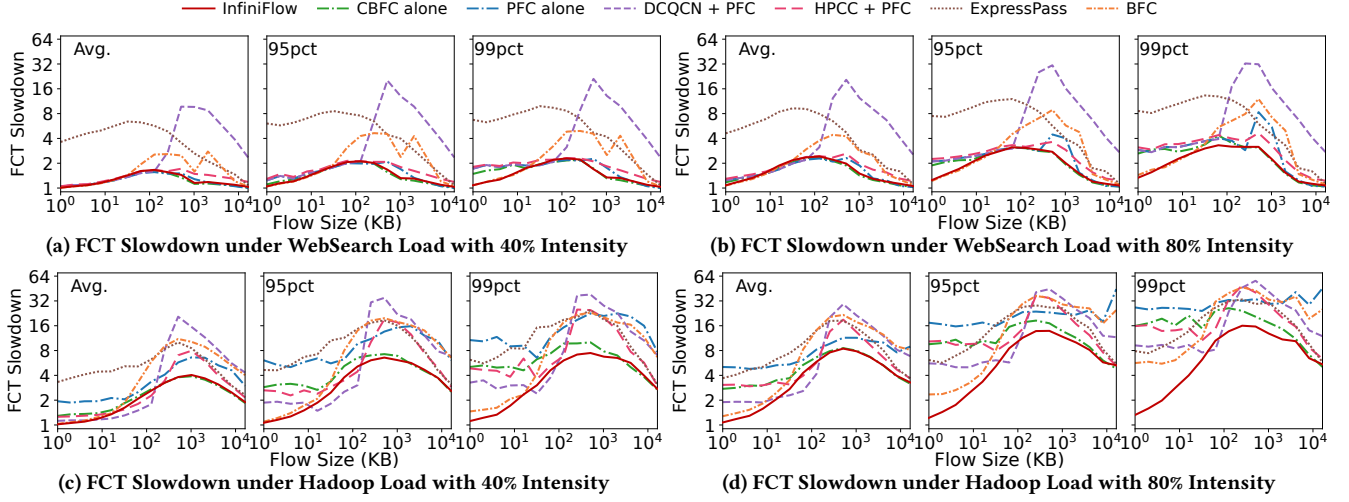


Figure 16: FCT Slowdown Comparison under Different Workloads for Various Transmission Methods

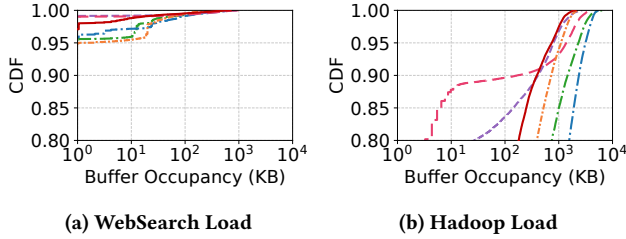


Figure 17: Buffer Occupancy Comparison under Different Workloads with 80% Intensity for Various Transmission Methods, sharing the legend with Figure 16

Tail FCT for Short Flows. Across most workloads, InfiniFlow achieves the lowest tail slowdown for short flows, with the largest gains under heavy load (80%) and the long-flow-heavy Hadoop workload, where PFC/CBFC and BFC incur severe tail inflation from VC contention. This reflects InfiniFlow’s ability to reduce VC collisions by supporting massive VCs. Compared to standalone flow control, CC schemes can reduce short-flow tail FCT, but RTT-bounded feedback and rate oscillations prevent them from achieving the best tail FCT.

Average FCT for Long Flows. For large flows, InfiniFlow also maintains low average FCT across most settings, showing that hop-by-hop control with sufficient isolation keeps long flows near fair-share rates while fully utilizing bandwidth. In contrast, PFC/CBFC are comparable only under 40% load and degrade at high load as limited VCs exacerbate HoLB. BFC performs poorly due to excessive retransmissions. Although CC schemes reduce tail FCTs, their average FCTs are worse than standalone flow control, suggesting underutilized bandwidth.

Buffer Occupancy. Finally, we examine overall buffer occupancy at 80% load (Figure 17): under both workloads, standalone PFC and CBFC build the largest backlog due to congestion spreading. In contrast, InfiniFlow shifts the CDF noticeably left by minimizing buffer monopolization. BFC improves over PFC/CBFC but still lags InfiniFlow. Although CCs can reduce occupancy even better than InfiniFlow, their smaller backlogs do not translate into the best FCT performance.

7 Discussion

Scheduling Beyond Max-Min Fairness. We use global max-min fair transmission as the ideal baseline. This does not imply that InfiniFlow is limited to max-min fairness. In fact, max-min fairness arises from the use of an equal-weight DRR scheduler. More broadly, massive VC scalability opens up design space for alternative packet scheduling policies.

Beyond Per-Hop BDP Buffer Requirements. As DCNs scale, longer fiber runs [46] and higher link rates [35] increase the per-hop BDP, raising the buffer space requirement. Under the same buffer budget, InfiniFlow can tolerate larger per-hop BDPs than traditional methods. However, future DCNs may push the per-hop BDP beyond what InfiniFlow can accommodate, motivating BDP-independent lossless flow control as a direction for future work.

Functional Boundaries of InfiniFlow. InfiniFlow shifts the rate adjustment role of L4 CC to L2 flow control, while preventing congestion-induced losses. However, InfiniFlow cannot eliminate losses caused by device failures; therefore, end hosts still need L4 protocols to ensure reliable delivery. InfiniFlow operates independently of L4 retransmission mechanisms: original packets, retransmitted packets, and ACKs consume and release link-local credits in the same manner. In addition, InfiniFlow is orthogonal to L3 routing. How to co-design InfiniFlow with load balancers beyond ECMP is another interesting direction for future work.

Failure Handling Mechanism. For an L2 flow control mechanism, each link operates as a fate-sharing unit monitored via hardware keepalive signals. For each link, when an upstream node fails, the downstream node invalidates the corresponding flow control state, stops generating feedback on that link, and releases any associated local state after the failure is confirmed; when a downstream node fails, the upstream node detects the disconnection, disables transmission on the affected port, invalidates credits and per-VC states, and flushes packets from the corresponding port to reclaim buffer space and prevent deadlock. The end-to-end reliability is handled by L4 protocols.

Interactions among Per-Link Control Loops. In the current design of InfiniFlow, packet buffers remain statically partitioned

and isolated across ports, which limits direct interactions among the control loops of different ports on the same switch. However, multiple per-link control loops can still be indirectly coupled through a common downstream egress scheduler. Such coupling may introduce transient effects under rapid traffic changes. First, when congestion suddenly disappears, overly small parameter settings may cause transient under-utilization on some links. Second, when severe congestion suddenly appears, overly large parameter settings may allow congested VCs to retain more credits, and the slow packet draining rate can delay their release back to the shared credit pool for use by other VCs. To mitigate these issues, adaptive tuning of q_{min} and q_{max} that balances responsiveness and stability is an interesting direction for future work.

8 Related Work

Flow Control for DCNs. Beyond PFC, CBFC, and BFC, several recent DCN flow control mechanisms have been proposed. GFC [37] avoids coarse-grained pausing by gently regulating per-port rates to prevent deadlocks. OFC [44] and Aquarius [28] improve isolation by identifying the root of congestion and selectively applying backpressure only to the offending flows, preserving throughput for innocent traffic.

Congestion Control for DCNs. Recent DCN CC research focuses on improving control precision and convergence speed. ECN-based schemes [4, 13, 48] signal congestion via queue-based marking, which is echoed back for rate control. To better handle bursty traffic, INT-based designs [2, 7, 30] embed fine-grained link-load metadata in packets and return it via ACKs, enabling more accurate rate computation. RTT-based approaches [24, 33] instead infer congestion from end-host RTT fluctuations. Beyond sender-driven CCs, receiver-driven CCs [10, 18, 20, 34, 36] target incast by scheduling before injection to avoid oversubscription.

Switch Buffer Sharing. To elevate memory utilization, commodity switches widely employ shared buffer architectures [9]. DT [11] allocates queue thresholds proportional to the remaining buffer space, achieving fair allocation. Based on this, ABM [1] and Protean [5] incorporate adaptive threshold adjustment or selectively drop packets based on real-time traffic patterns and tenant isolation requirements.

9 Conclusion

We present InfiniFlow, an upstream-driven, credit-based flow control mechanism that supports massive VCs under a limited buffer budget. FPGA prototype evaluations and simulations show that InfiniFlow sustains line rate with a practical buffer budget while scaling to 16k+ VCs and delivering near-ideal FCTs on realistic workloads. InfiniFlow makes massive-VC lossless flow control practical and moves DCNs closer to optimal transmission.

Acknowledgments

We sincerely thank our shepherd and all reviewers for their insightful and constructive feedback. This work is sponsored by the National Natural Science Foundation of China (No. U25B2032, No. 62572128).

References

- [1] Vamsi Addanki, Maria Apostolaki, Manya Ghobadi, Stefan Schmid, and Laurent Vanbever. 2022. ABM: Active Buffer Management in Datacenters. In *Proceedings of the ACM SIGCOMM 2022 Conference*. 36–52. doi:10.1145/3544216.3544252
- [2] Vamsi Addanki, Oliver Michel, and Stefan Schmid. 2022. PowerTCP: Pushing the Performance Limits of Datacenter Networks. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. USENIX Association, Renton, WA, 51–70. <https://www.usenix.org/conference/nsdi22/presentation/addanki>
- [3] Advanced Micro Devices, Inc. n.d.. AMD Alveo™ U280 Data Center Accelerator Card (Product Brief). <https://www.xilinx.com/publications/product-briefs/alveo-u280-product-brief.pdf>. Official product brief for the Alveo U280 accelerator card.
- [4] Mohammad Alizadeh, Albert Greenberg, David A. Maltz, Jitendra Padhye, Parveen Patel, Balaji Prabhakar, Sudipta Sengupta, and Murari Sridharan. 2010. Data center TCP (DCTCP). *SIGCOMM Comput. Commun. Rev.* 40, 4 (Aug. 2010), 63–74. doi:10.1145/1851275.1851192
- [5] Hamidreza Almasi, Rohan Vardekar, and Balajee Vamanan. 2023. Protean: Adaptive Management of Shared-Memory in Datacenter Switches. In *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications*. 1–10. doi:10.1109/INFOCOM53939.2023.10229046
- [6] Arista Networks. n.d.. Arista 7800R3 Platform Architecture. <https://www.arista.com/assets/data/pdf/Whitepapers/Arista7800R3SwitchArchitectureWP.pdf>. Reports on-chip buffers of 32MB with Jericho2 and 64MB with Jericho2C+.
- [7] Ran Ben Basat, Sivaramakrishnan Ramanathan, Yuliang Li, Gianni Antichi, Minian Yu, and Michael Mitzenmacher. 2020. PINT: Probabilistic In-band Network Telemetry. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication (Virtual Event, USA) (SIGCOMM '20)*. Association for Computing Machinery, New York, NY, USA, 662–680. doi:10.1145/3387514.3405894
- [8] Dimitri P. Bertsekas and Robert G. Gallager. 1992. *Data Networks* (2nd ed.). Prentice Hall, Upper Saddle River, NJ, USA. <https://web.mit.edu/dimitrib/www/datanets.html>
- [9] Broadcom. n.d.. Broadcom Trident 3 Platform Performance Analysis. <https://docs.broadcom.com/docs/12395356>. States Trident3 MMU is designed with 32MB integrated, fully shared buffer memory.
- [10] Inho Cho, Keon Jang, and Dongsu Han. 2017. Credit-Scheduled Delay-Bounded Congestion Control for Datacenters. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication (Los Angeles, CA, USA) (SIGCOMM '17)*. Association for Computing Machinery, New York, NY, USA, 239–252. doi:10.1145/3098822.3098840
- [11] Abhijit K. Choudhury and Ellen L. Hahne. 1998. Dynamic queue length thresholds for shared-memory packet switches. *IEEE/ACM Transactions on Networking* 6, 2 (1998), 130–140. doi:10.1109/90.664262
- [12] Cisco Systems. 2024. Cisco 100GBASE QSFP-100G Modules Data Sheet. <https://www.cisco.com/c/en/us/products/collateral/interfaces-modules/transceiver-modules/datasheet-c78-736282.html>. QSFP-100G-SR4-S supports up to 70 m over OM3 and up to 100 m over OM4 multimode fiber.
- [13] Xinle Du, Tong Li, Guangmeng Zhou, Zhuotao Liu, Hanlin Huang, Xiangyu Gao, Mowei Wang, Kun Tan, and Ke Xu. 2025. PRED: Performance-oriented Random Early Detection for Consistently Stable Performance in Datacenters. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. USENIX Association, Philadelphia, PA, 1–20. <https://www.usenix.org/conference/nsdi25/presentation/du>
- [14] Robert Elz and Randy Bush. 1996. RFC 1982: Serial Number Arithmetic. <https://datatracker.ietf.org/doc/html/rfc1982>. Defines wrap-around-safe arithmetic and comparison rules for serial numbers in a finite modulo space.
- [15] Alex Forencich, Alex C. Snoeren, George Porter, and George Papen. 2020. Corundum: An Open-Source 100-Gbps NIC. In *Proceedings of the 2020 28th IEEE International Symposium on Field-Programmable Custom Computing Machines (Fayetteville, AR, USA) (FCCM '20)*. IEEE, Piscataway, NJ, USA, 38–46. doi:10.1109/FCCM48280.2020.00015
- [16] Prateesh Goyal, Preet Shah, Kevin Zhao, Georgios Nikolaidis, Mohammad Alizadeh, and Thomas E. Anderson. 2022. Backpressure Flow Control. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. USENIX Association, Renton, WA, 779–805. <https://www.usenix.org/conference/nsdi22/presentation/goyal>
- [17] IEEE 802.1 Working Group. 2011. IEEE Standard for Local and Metropolitan Area Networks—Virtual Bridged Local Area Networks Amendment: Priority-based Flow Control. IEEE 802.1Qbb Project Page. Standard approved June 16, 2011. <https://1.ieee802.org/dcb/802-1qbb>
- [18] Mark Handley, Costin Raiciu, Alexandru Agache, Andrei Voinescu, Andrew W. Moore, Gianni Antichi, and Marcin Wójcik. 2017. Re-architecting datacenter networks and stacks for low latency and high performance. (2017), 29–42. doi:10.1145/3098822.3098825

- [19] Jinbin Hu, Chaoliang Zeng, Zilong Wang, Hong Xu, Jiawei Huang, and Kai Chen. 2023. Load Balancing in PFC-Enabled Datacenter Networks. In *Proceedings of the 6th Asia-Pacific Workshop on Networking* (Fuzhou, China) (APNet '22). Association for Computing Machinery, New York, NY, USA, 21–28. doi:10.1145/3542637.3542641
- [20] Shuihai Hu, Wei Bai, Gaoxiong Zeng, Zilong Wang, Baochen Qiao, Kai Chen, Kun Tan, and Yi Wang. 2020. Aeolus: A Building Block for Proactive Transport in Datacenters. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication* (Virtual Event, USA) (SIGCOMM '20). Association for Computing Machinery, New York, NY, USA, 422–434. doi:10.1145/3387514.3405878
- [21] INET-TUB. 2025. ns3-datacenter. <https://github.com/inet-tub/ns3-datacenter>. GitHub repository.
- [22] Saurabh Jha, Archit Patke, Jim Brandt, Ann Gentile, Benjamin Lim, Mike Showerman, Greg Bauer, Larry Kaplan, Zbigniew Kalbarczyk, William Kramer, and Ravi Iyer. 2020. Measuring Congestion in High-Performance Datacenter Interconnects. In *17th USENIX Symposium on Networked Systems Design and Implementation* (NSDI 20). USENIX Association, Santa Clara, CA, 37–57. <https://www.usenix.org/conference/nsdi20/presentation/jha>
- [23] Anuj Kalia, Michael Kaminsky, and David Andersen. 2019. Datacenter RPCs can be General and Fast. In *16th USENIX Symposium on Networked Systems Design and Implementation* (NSDI 19). USENIX Association, Boston, MA, 1–16. <https://www.usenix.org/conference/nsdi19/presentation/kalia>
- [24] Gautam Kumar, Nandita Dukkipati, Keon Jang, Hassan M. G. Wassel, Xian Wu, Behnam Montazeri, Yaogong Wang, Kevin Springborn, Christopher Alfeld, Michael Ryan, David Wetherall, and Amin Vahdat. 2020. Swift: Delay is Simple and Effective for Congestion Control in the Datacenter. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication* (Virtual Event, USA) (SIGCOMM '20). Association for Computing Machinery, New York, NY, USA, 514–528. doi:10.1145/3387514.3406591
- [25] Hsiang-Tsung Kung, Trevor Blackwell, and Alan Chapman. 1994. Credit-based flow control for ATM networks: credit update protocol, adaptive credit allocation and statistical multiplexing. In *Proceedings of the Conference on Communications Architectures, Protocols and Applications* (London, United Kingdom) (SIGCOMM '94). Association for Computing Machinery, New York, NY, USA, 101–114. doi:10.1145/190314.190324
- [26] ChonLam Lao, Yanfang Le, Kshiteej Mahajan, Yixi Chen, Wenfei Wu, Aditya Akella, and Michael Swift. 2021. ATP: In-network Aggregation for Multi-tenant Learning. In *18th USENIX Symposium on Networked Systems Design and Implementation* (NSDI 21). USENIX Association, 741–761. <https://www.usenix.org/conference/nsdi21/presentation/lao>
- [27] Hugh C. Lauer. 1995. *On the Duality of Rate-based and Credit-based Flow Control*. Technical Report. Mitsubishi Electric Research Laboratories (MERL).
- [28] Wenxue Li, Chaoliang Zeng, Jinbin Hu, and Kai Chen. 2023. Scaling Switch-driven Flow Control with Aquarius. In *Proceedings of the 7th Asia-Pacific Workshop on Networking* (Hong Kong, China) (APNet '23). Association for Computing Machinery, New York, NY, USA, 81–87. doi:10.1145/3600061.3600066
- [29] Wenxue Li, Chaoliang Zeng, Jinbin Hu, and Kai Chen. 2023. Towards Fine-Grained and Practical Flow Control for Datacenter Networks. In *2023 IEEE 31st International Conference on Network Protocols (ICNP)*. 1–11. doi:10.1109/ICNP59255.2023.10355582
- [30] Yuliang Li, Rui Miao, Hongqiang Harry Liu, Yan Zhuang, Fei Feng, Lingbo Tang, Zheng Cao, Ming Zhang, Frank Kelly, Mohammad Alizadeh, and Minlan Yu. 2019. HPCC: high precision congestion control. In *Proceedings of the ACM Special Interest Group on Data Communication* (Beijing, China) (SIGCOMM '19). Association for Computing Machinery, New York, NY, USA, 44–58. doi:10.1145/3341302.3342085
- [31] Kexin Liu, Zhaochen Zhang, Chang Liu, Yizhi Wang, Vamsi Addanki, Stefan Schmid, Qingyue Wang, Wei Chen, Xiaoliang Wang, Jiaqi Zheng, Wenhao Sun, Tao Wu, Ke Meng, Fei Chen, Weiguang Wang, Bingyang Liu, Wanchun Dou, Guihai Chen, and Chen Tian. 2025. Pyrrha: Congestion-Root-Based Flow Control to Eliminate Head-of-Line Blocking in Datacenter. In *22nd USENIX Symposium on Networked Systems Design and Implementation* (NSDI 25). USENIX Association, Philadelphia, PA, 379–405. <https://www.usenix.org/conference/nsdi25/presentation/liu-kexin>
- [32] Mellanox Technologies (NVIDIA Networking). 2018. 100Gb/s EDR SR4 QSFP28 MMF Optical Transceiver (Product Brief): MMA1B00-E100. https://network.nvidia.com/related-docs/prod_cables/PB_MMA1B00-E100_100Gbps_QSFP28_MMF_Transceiver.pdf. States typical reach: 100 m on OM4 and 70 m on OM3 at 100 Gb/s.
- [33] Radhika Mittal, Vinh The Lam, Nandita Dukkipati, Emily Blem, Hassan Wassel, Monia Ghobadi, Amin Vahdat, Yaogong Wang, David Wetherall, and David Zats. 2015. TIMELY: RTT-based Congestion Control for the Datacenter. *SIGCOMM Comput. Commun. Rev.* 45, 4 (Aug. 2015), 537–550. doi:10.1145/2829988.2787510
- [34] Behnam Montazeri, Yilong Li, Mohammad Alizadeh, and John Ousterhout. 2018. Homa: a receiver-driven low-latency transport protocol using network priorities. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication* (Budapest, Hungary) (SIGCOMM '18). Association for Computing Machinery, New York, NY, USA, 221–235. doi:10.1145/3230543.3230564
- [35] Leon Poutievski, Omid Mashayekhi, Joon Ong, Arjun Singh, Mukarram Tariq, Rui Wang, Jianan Zhang, Virginia Beauregard, Patrick Conner, Steve Gribble, Rishi Kapoor, Stephen Kratzer, Nanfang Li, Hong Liu, Karthik Nagaraj, Jason Ornstein, Samir Sawhney, Ryohei Urata, Lorenzo Vicisano, Kevin Yasumura, Shidong Zhang, Junlan Zhou, and Amin Vahdat. 2022. Jupiter evolving: transforming google's datacenter network via optical circuit switches and software-defined networking. In *Proceedings of the ACM SIGCOMM 2022 Conference* (Amsterdam, Netherlands) (SIGCOMM '22). Association for Computing Machinery, New York, NY, USA, 66–85. doi:10.1145/3544216.3544265
- [36] Konstantinos Prasopoulos, Ryan Kosta, Edouard Bugnion, and Marios Kogias. 2025. SIRD: A Sender-Informed, Receiver-Driven Datacenter Transport Protocol. In *22nd USENIX Symposium on Networked Systems Design and Implementation* (NSDI 25). USENIX Association, Philadelphia, PA, 451–471. <https://www.usenix.org/conference/nsdi25/presentation/prasopoulos>
- [37] Kun Qian, Wenxue Cheng, Tong Zhang, and Fengyuan Ren. 2019. Gentle flow control: avoiding deadlock in lossless networks. In *Proceedings of the ACM Special Interest Group on Data Communication* (Beijing, China) (SIGCOMM '19). Association for Computing Machinery, New York, NY, USA, 75–89. doi:10.1145/3341302.3342065
- [38] Stephen M. Rumble, Diego Ongaro, Ryan Stutsman, Mendel Rosenblum, and John K. Ousterhout. 2011. It's Time for Low Latency. In *13th Workshop on Hot Topics in Operating Systems (HotOS XIII)*. USENIX Association, Napa, CA. <https://www.usenix.org/conference/hotosxiii/its-time-low-latency>
- [39] Jose Renato Santos, Yoshio Turner, and G. John Janakiraman. 2003. End-to-end congestion control for infiniband. In *IEEE INFOCOM 2003. Twenty-second Annual Joint Conference of the IEEE Computer and Communications Societies (IEEE Cat. No.03CH37428)*, Vol. 2. 1123–1133 vol.2. doi:10.1109/INFCOM.2003.1208949
- [40] Amedeo Sapia, Marco Canini, Chen-Yu Ho, Jacob Nelson, Panos Kalnis, Changhoon Kim, Arvind Krishnamurthy, Masoud Moshref, Dan Ports, and Peter Richtárik. 2021. Scaling distributed machine learning with In-Network aggregation. In *18th USENIX Symposium on Networked Systems Design and Implementation* (NSDI 21). 785–808.
- [41] Mellanox Technologies. 2014. InfiniBand Credit-Based Link-Layer Flow Control. IEEE 802.1 DCB Task Group Tutorial. IEEE 802 Plenary Presentation. <https://groupieee.org/groups/802/1/files/public/docs2014/new-dcb-crupnicoff-ibcreditstutorial-0314.pdf>
- [42] Balajee Vamanan, Jahangir Hasan, and T. N. Vijaykumar. 2012. Deadline-aware datacenter tcp (D2TCP). In *Proceedings of the ACM SIGCOMM 2012 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication* (Helsinki, Finland) (SIGCOMM '12). Association for Computing Machinery, New York, NY, USA, 115–126. doi:10.1145/2342356.2342388
- [43] Weitao Wang, Masoud Moshref, Yuliang Li, Gautam Kumar, T. S. Eugene Ng, Neal Cardwell, and Nandita Dukkipati. 2023. Poseidon: Efficient, Robust, and Practical Datacenter CC via Deployable INT. In *20th USENIX Symposium on Networked Systems Design and Implementation* (NSDI 23). USENIX Association, Boston, MA, 255–274. <https://www.usenix.org/conference/nsdi23/presentation/wang-Weitao>
- [44] Wenli Xiao, Yuqing Yang, Peirui Cao, Zhuoran Liu, Shizhen Zhao, and Xinbing Wang. 2024. OFC: An Original Congestion-Based Fine-grained Priority Flow Control. In *2024 21st Annual IEEE International Conference on Sensing, Communication, and Networking (SECON)*. 28–36. doi:10.1109/SECON64284.2024.10934869
- [45] Shihang Yan, Geyong Min, and Irfan Awan. 2006. Performance Analysis of Credit-Based Flow Control in InfiniBand Interconnection Networks. *Journal of Interconnection Networks* 7 (12 2006), 535–548. doi:10.1142/S0219265906001843
- [46] Peiwen Yu, Feiyang Xue, Chen Tian, Xiaoliang Wang, Yanqing Chen, Tao Wu, Lei Han, Zifa Han, Bingquan Wang, Xiangyu Gong, Wanchun Dou, and Guihai Chen. 2023. Bifrost: Extending RoCE for Long Distance Inter-DC Links. In *2023 IEEE 31st International Conference on Network Protocols (ICNP)*. 1–12. doi:10.1109/ICNP59255.2023.10355634
- [47] Yiran Zhang, Qingkai Meng, Chaolei Hu, and Fengyuan Ren. 2024. Revisiting Congestion Control for Lossless Ethernet. In *21st USENIX Symposium on Networked Systems Design and Implementation* (NSDI 24). USENIX Association, Santa Clara, CA, 131–148. <https://www.usenix.org/conference/nsdi24/presentation/zhang-yiran>
- [48] Yibo Zhu, Haggai Eran, Daniel Firestone, Chuanxiong Guo, Marina Lipshteyn, Yehonatan Liron, Jitendra Padhye, Shachar Raindel, Mohamad Haj Yahia, and Ming Zhang. 2015. Congestion Control for Large-Scale RDMA Deployments. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication* (London, United Kingdom) (SIGCOMM '15). Association for Computing Machinery, New York, NY, USA, 523–536. doi:10.1145/2785956.2787484

Appendix

Appendices are supporting material that has not been peer-reviewed.

A Proof of Max-Min Fairness for Per-Flow Flow Control

In this part, we prove that per-flow CBFC or PFC networks equipped with DRR schedulers achieve global max-min fairness in steady state, assuming the buffer space of each VC is sufficient to sustain line-rate lossless transmission. First, let us focus on CBFC. This proof includes (1) the notations and definitions, (2) a key lemma, and (3) the main theorem.

A.1 Notations and Definitions

Consider a network $G = (V, L)$ with a set of flows F .

Variables.

- C_l : physical capacity of link $l \in L$.
- x_f : steady-state sending rate of flow $f \in F$.
- P_f : ordered set of links on the path of flow f .
- S_l : set of flows traversing link l , $S_l = \{f \in F \mid l \in P_f\}$.
- $\vec{x}$: global rate vector, $\vec{x} = (x_f)_{f \in F}$.
- $\mathcal{X}$: set of all feasible rate vectors.

Feasibility. A rate vector $\vec{x}$ is *feasible* if it satisfies the non-negativity constraints and the capacity constraints for all links. The set of feasible rate vectors $\mathcal{X}$ is defined as:

$$\mathcal{X} = \left\{ \vec{x} \mid x_f \geq 0, \forall f \in F \text{ and } \sum_{f \in S_l} x_f \leq C_l, \forall l \in L \right\} \quad (7)$$

Max-Min Fairness. A feasible rate vector $\vec{x} \in \mathcal{X}$ is *max-min fair* if it is impossible to increase the rate of any flow f without decreasing the rate of another flow f' that has an equal or smaller rate. Formally, $\vec{x}$ is max-min fair if and only if, for any other feasible vector $\vec{y} \in \mathcal{X}$: if $\exists f \in F$ such that $y_f > x_f$, then $\exists f' \in F$ such that $y_{f'} < x_{f'}$ and $x_{f'} \leq x_f$.

Steady-state predicates. In steady state we use the following predicates:

- *Backlogged*(f, l): the VC queue of f at link l is persistently non-empty.
- *HasCredit*(f, l): f has non-zero credit balance at link l in steady state.

We say that a flow is *persistently eligible* (PE) at link l if it continuously satisfies both conditions above:

$$\text{PE}(f, l) \triangleq \text{Backlogged}(f, l) \wedge \text{HasCredit}(f, l).$$

In practice, $\text{PE}(f, l) = \text{True}$ implies that flow f permanently resides in the active list of the scheduler at link l .

Property of DRR. DRR ensures that, over the long term, any set of flows that remain persistently eligible at the same link receive equal steady-state service (assuming equal weights):

$$\forall i, j \in S_l, \quad \text{PE}(i, l) \wedge \text{PE}(j, l) \Rightarrow x_i = x_j.$$

A.2 Lemma: The PE Bottleneck

LEMMA A.1. *For any flow $f \in F$, there exists a link $l^* \in P_f$ such that f is persistently eligible at l^* .*

PROOF. Due to the backpressure mechanism of CBFC, congestion propagates upstream. Therefore, the link set where flow f is backlogged forms a contiguous segment ending at the effective bottleneck; links downstream of this point are faster than the arrival rate and thus have empty queues.

Definition of l^* . We define l^* to be the *downstream-most* link on P_f where flow f is backlogged. Then we prove f is persistently eligible at l^* . By the definition of l^* , the Backlogged condition is automatically satisfied.

Credit condition. We now verify that f has credits at l^* . Suppose, for the sake of contradiction, that f has no credit at l^* . Under CBFC, a lack of credits implies that flow f is blocked by the receiver of the downstream link due to buffer filling. This necessitates that flow f is also backlogged at the subsequent downstream link. However, this contradicts our definition of l^* as the *downstream-most* backlogged link. Therefore, the assumption is false.

In summary, at the *downstream-most* backlogged link l^* , flow f is backlogged and holds credits. Thus:

$$\text{PE}(f, l^*) = \text{True}.$$

□

We define the link l^* as the PE bottleneck of flow f .

A.3 Main Theorem

THEOREM A.2. *In a network that employs per-flow CBFC and DRR scheduling with equal weights, the steady-state rate vector $\vec{x}$ is max-min fair.*

PROOF. By the necessary and sufficient conditions of Bertsekas and Gallager [8], it suffices to show that for every flow $f \in F$ there exists a bottleneck link $l^* \in P_f$ such that:

(1) **Saturation:**

$$\sum_{k \in S_{l^*}} x_k = C_{l^*}.$$

(2) **Maximality:**

$$x_f \geq x_k, \quad \forall k \in S_{l^*}.$$

Step 1: Selecting l^ .* By Lemma A.1, pick the PE bottleneck l^* of f , where f is persistently eligible, i.e.,

$$\text{PE}(f, l^*) = \text{True}.$$

Step 2: Proving Saturation. DRR is work-conserving with respect to persistently eligible flows. Since f is persistently eligible, f permanently resides in the active list of the DRR scheduler at l^* . Thus the link is fully utilized:

$$\sum_{k \in S_{l^*}} x_k = C_{l^*}.$$

Step 3: Proving Maximality. Partition flows on l^* into:

$$E_{l^*} = \{k \in S_{l^*} \mid \text{PE}(k, l^*)\}, \quad U_{l^*} = S_{l^*} \setminus E_{l^*}.$$

Case A: $k \in E_{l^*}$. All persistently eligible flows share equal service under DRR:

$$x_k = x_f.$$

Case B: $k \in U_{l^*}$. Flow k is constrained elsewhere along its path and cannot exploit the fair share available at l^* , implying:

$$x_k < x_f.$$

Therefore,

$$\forall k \in S_{l^*}, \quad x_f \geq x_k,$$

and the bottleneck conditions hold. Hence, according to the proposition in §6.5 of *Data Networks* [8], $\bar{x}$ is max-min fair. $\square$

A.4 Extension to per-flow PFC Networks

The theoretical framework established above for CBFC can be directly extended to networks utilizing per-flow PFC.

In a PFC network, flow control operates via XOFF (Pause) and XON (Resume) signaling. Let *NotPaused*(f, l) denote the condition that flow f is free from PFC restrictions at link l in the steady state. Specifically, this means that the downstream node does not assert XOFF signals for flow f , allowing the scheduler at link l to transmit whenever it chooses.

We define the complementary state as:

$$\text{Paused}(f, l) \triangleq \neg \text{NotPaused}(f, l).$$

Note that *Paused*(f, l) = True does not imply zero throughput; rather, it indicates that the flow is *throttled* by downstream backpressure (receiving intermittent XOFF signals).

Accordingly, the condition for a flow being persistently eligible becomes:

$$\text{PE}_{\text{PFC}}(f, l) \triangleq \text{Backlogged}(f, l) \wedge \text{NotPaused}(f, l).$$

This definition aligns with the bottleneck identification logic in Lemma A.1:

- If *Backlogged*(f, l) $\wedge$ *Paused*(f, l): Flow f has data but is throttled by downstream congestion signals. Thus, l is not the bottleneck.
- If *Backlogged*(f, l) $\wedge$ *NotPaused*(f, l): Flow f has data and is free to send, but its queue remains non-empty. This implies the local scheduler at l is the limiting factor. Thus, l is the bottleneck.

Since the DRR scheduler is work-conserving with respect to unpaused flows, the rest of the proof holds.

B InfiniFlow Logic Pseudo-code

In § 4, we elaborated on the specific mechanisms and design rationale of InfiniFlow. In this appendix, we provide a comprehensive walkthrough of the InfiniFlow packet transmission and reception workflows from a device-centric perspective, presented in the form of pseudocode. The implementation logic is organized into four components: (1) upstream data packet transmission; (2) downstream data packet processing; (3) downstream BCP generation; and (4) upstream BCP handling. In the pseudocode, we abbreviate the upstream FSM states Upstream Normal, Threshold Adjusting, and

Algorithm 1: Upstream Packet Transmission

```
function TransmitDataPkt():
    while (true):
        # check eligibility of each VC
        for each VC_i:
            cond1 = (VC_i.nxt != NULL)
            cond2 = (VC_i.nxt.len <= credit_pool)
            cond3 = (VC_i.nxt.len < threshold[i] -
                    inflight_bytes[i])
            eligible[i] = cond1 and cond2 and cond3
        # pick one eligible VC
        pkt, vc_id = scheduler(eligible)
        if (pkt != NULL):
            # send a TA-marked data packet
            if (state[vc_id] == TA):
                pkt.TA_FLAG = true
                state[vc_id] = WT
            # maintain counters and variables
            VC_TX[vc_id] += pkt.len
            inflight_bytes[vc_id] =
                VC_TX[vc_id] - bcp.VC_DR
            PT_TX += pkt.len
            pkt.ingress_port.VC_DR[vc_id] += pkt.len
            credit_pool -= pkt.len
            # send a BCP from pkt's ingress port
            pkt.ingress_port.TransmitBCP(vc_id)
            # transmit data packet
            send(pkt)
```

Algorithm 2: Downstream Packet Processing

```
function ReceiveDataPkt(pkt):
    # extract VC id
    vc_id = pkt.VC
    # maintain counters and variables
    PT_RX += pkt.len
    VC_RX[vc_id] += pkt.len
    # TA flag handling
    if (pkt.TA_FLAG):
        state[vc_id] = FM
    # forward to egress pipeline
    to_egress(pkt)
```

Feedback Waiting as UN, TA, and WT, respectively, and the downstream FSM states Downstream Normal and Feedback Marking as DN and FM, respectively.

We initiate our walkthrough from the egress pipeline of the upstream node. The upstream node schedules packets from the egress port, updating the related counters and variables before transmission (Algorithm 1).

Upon receiving a data packet, the downstream node updates counters, caches the packet in the shared buffer and forwards the packet pointer to its destination port (Algorithm 2).

The departure of a data packet from the shared buffer triggers the generation of a BCP to provide feedback to the upstream node (Algorithm 3).

The upstream node updates its variables and adjusts thresholds upon receiving a BCP (Algorithm 4).

Algorithm 3: Downstream BCP Generation

```

function TransmitBCP(vc_id):
    # generate a BCP packet
    bcp = get_empty_bcp()
    bcp.VC = vc_id
    bcp.PT_FCCL = RSBS + PT_RX
    bcp.VC_BKLG = VC_RX[vc_id] - VC_DR[vc_id]
    bcp.VC_DR = VC_DR[vc_id]
    # return a TA-marked BCP if in FM state
    if (state[vc_id] == FM):
        bcp.TA_FLAG = true
        state[vc_id] = DN
    # send the BCP
    send(bcp)

```

Algorithm 4: Upstream BCP Handling

```

function ReceiveBCP(bcp):
    # extract VC id
    vc_id = bcp.VC
    # update counters and variables
    inflight_bytes[vc_id] = VC_TX[vc_id] - bcp.VC_DR
    PT_FCCL = bcp.PT_FCCL
    credit_pool = PT_FCCL - PT_TX
    # TA flag handling
    if (bcp.TA_FLAG == true):
        state[vc_id] = UN
    # threshold adjusting logic
    if (state[vc_id] == UN):
        if (bcp.VC_BKLG >= qmax):
            threshold[vc_id] = max(MTU+1,
                inflight_bytes[vc_id] - (bcp.VC_BKLG - qmin))
            state[vc_id] = TA
        else if (bcp.VC_BKLG < qmin):
            threshold[vc_id] = min(RSBS,
                threshold[vc_id] + qmin - bcp.VC_BKLG)
            state[vc_id] = TA

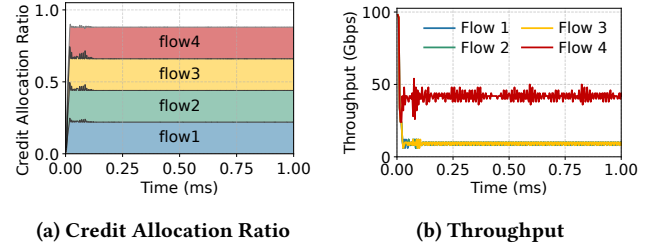
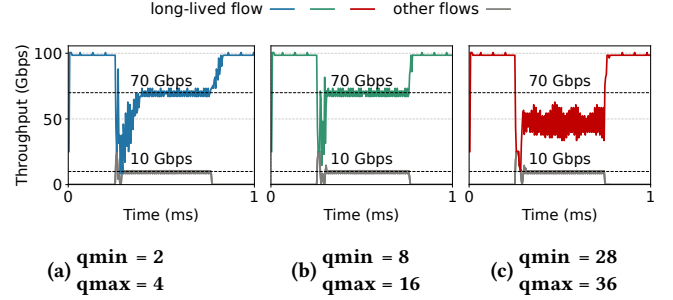
```

C Limitations of DT-based Credit Allocation

Throttled VCs can rapidly deplete the shared buffer, starving innocent VCs [25]. Therefore, credit-based buffer sharing must rely on a credit allocation mechanism to re-establish inter-VC isolation. Intuitively, this can be addressed by dynamic reservation solutions, such as DT [11], which is a widely adopted technique in shared-memory switches. DT can *evenly allocate credits* among active VCs, preventing a throttled VC from monopolizing the shared buffer.

However, even allocation is insufficient when VCs have different demands. We demonstrate this via an experiment (Figure 18) with the same setting as Figure 14: a dumbbell topology where one victim flow (Flow 4, which experiences no congestion except at the bottleneck link) shares the bottleneck link (line rate = 100 Gbps, link delay = 5 μ s) with three congested flows (Flows 1–3, throttled to 10 Gbps by downstream links). The shared buffer size is around $2 \times$ per-hop BDP, and the DT parameter α is 2. Each flow is mapped to a distinct VC, and DT enforces fair credit allocation among these four flows as shown in Figure 18a.

Ideally, Flow 4 should fully utilize the spare bandwidth (70 Gbps). However, as shown in Figure 18b, it is capped at less than 50 Gbps.

**Figure 18: Limitations of DT-based Credit Allocation****Figure 19: Throughput of Long-Lived Flow under Three Typical Parameter Settings.**

This is because, by treating all active VCs fairly, DT restricts the high-speed victim flow's credit to the same level as the congested flows. This result underscores the need for a protocol that can allocate credit according to each VC's demand, ensuring that individual transmission demands are fully met while strictly preventing buffer abuse.

D Parameter Setting Analysis

InfiniFlow has two key parameters, q_{min} and q_{max} . In this appendix, we first use microbenchmarks to show the problems caused by overly small or overly large settings, and then conduct large-scale parameter sweeps to characterize how parameter choices affect overall performance.

D.1 Impact of Suboptimal Parameter Settings

Both overly small and overly large parameter settings can degrade InfiniFlow's performance. To understand the underlying causes, we evaluate InfiniFlow under three (q_{min} , q_{max}) settings: (2,4), (8,16), and (28,36), corresponding to overly small, moderate, and overly large settings, respectively, in units of MTUs. The topology and traffic setup are similar to those in Figure 14: a dumbbell topology where one long-lived flow (Flow 4, which experiences no congestion except at the bottleneck link) shares a 100 Gbps bottleneck link with three congested flows (Flows 1–3, throttled to 10 Gbps by downstream links). All links have a delay of 2 μ s, and the shared buffer size is approximately $3 \times$ per-hop BDP. Flows 1–3 start at around 250 μ s and finish at around 750 μ s. Figure 19 compares the rate of the long-lived flow under overly small and overly large parameter settings, respectively, against those under the moderate setting.

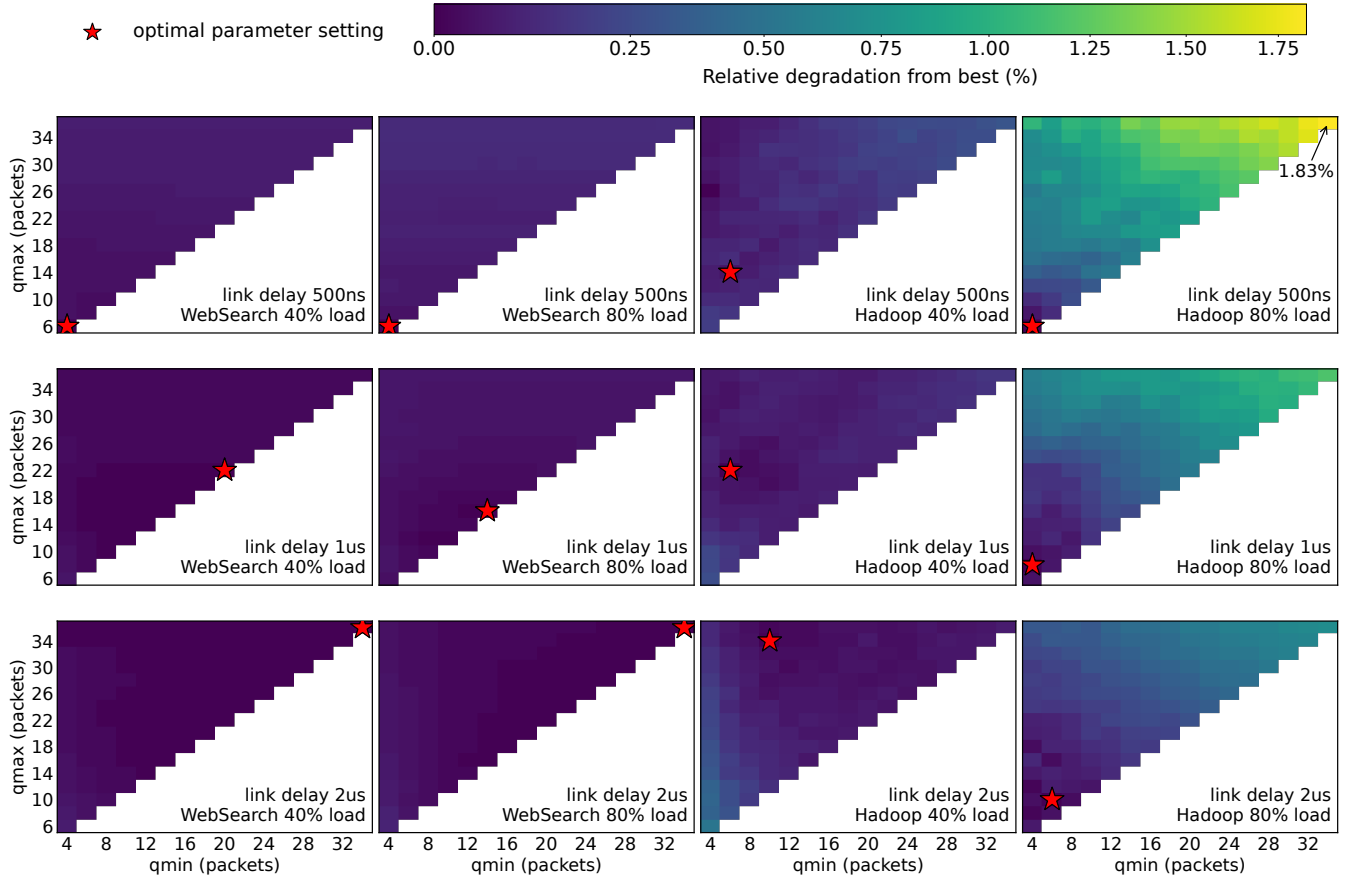


Figure 20: Average FCT Slowdown Sensitivity to Parameter Settings under Diverse Network Conditions.

With overly small parameter settings, as shown in Figure 19a, Flow 4 takes a relatively long time to adjust to its target rate of 70 Gbps around the start and finish of Flows 1–3. This is due to two reasons. First, according to BUCP’s per-VC credit threshold adjustment algorithm, the threshold increase rate is proportional to q_{min} . Second, overly small q_{min} and q_{max} imply that each flow maintains a smaller steady-state backlog. During the interval between the sudden disappearance of congestion and the full recovery of the transmission rate, such a small backlog cannot support no-bubble transmission, which is a side effect of maintaining an extremely low steady-state backlog. Although CBFC can achieve no-bubble transmission under this condition, it requires reserving one per-hop BDP of backlog in the buffer before congestion disappears.

With overly large parameter settings, as shown in Figure 19c, Flow 4 cannot reach the ideal rate of 70 Gbps when Flows 1–4 run concurrently. This is because overly large q_{min} and q_{max} cause Flows 1–3 to maintain excessive steady-state backlog, tying up a large amount of credits in the buffer. As a result, the remaining credits are insufficient to sustain Flow 4 at 70 Gbps.

D.2 Large-Scale Parameter Sweep Experiments

We further conduct large-scale parameter sweep experiments to evaluate the impact of q_{min} and q_{max} under diverse network conditions. We use a 1024-host leaf-spine topology with 100 Gbps links and vary three factors: workload type (WebSearch and Hadoop), load intensity (40% and 80%), and link delay (500 ns, 1 μ s, and 2 μ s). For each condition, we compute the relative gap between the average FCT slowdown under each parameter setting, denoted as AFS_x , and the minimum average FCT slowdown under the same condition, denoted as AFS_{min} , i.e., $(AFS_x - AFS_{min}) / AFS_{min}$. We then visualize the results as heatmaps. Each heatmap cell represents the relative performance degradation from the optimal setting, expressed as a percentage. All heatmaps share the same color scale. The optimal point is marked by a star. When multiple settings achieve the same optimum, we mark the one with the largest parameter values.

Figure 20 shows three main trends. **First, under the same workload**, increasing link delay generally shifts the optimal point toward larger parameter values. This indicates that a longer feedback delay requires a larger steady-state backlog to reduce transient bubbles. **Second, under the same delay and load intensity**, different application characteristics lead to different optimal settings.

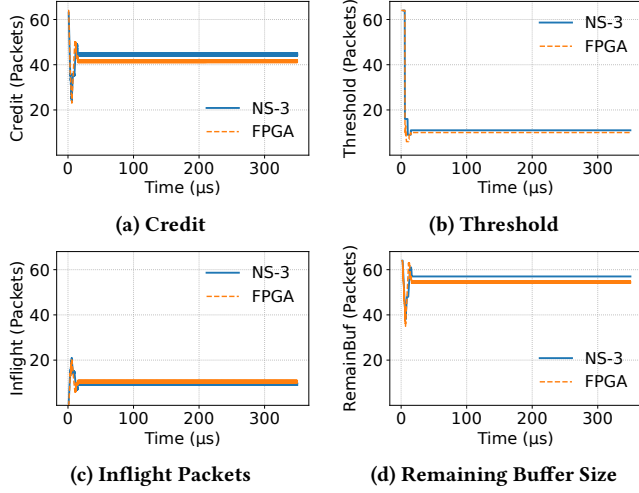


Figure 21: Validation of FPGA Prototype Fidelity against NS-3 Simulation under Congestion.

Compared with WebSearch, Hadoop usually prefers smaller parameter values, especially under 80% load, where the optimal point clearly shifts toward the lower-left region. This is because the long-flow-intensive Hadoop workload tends to create persistent buffer pressure, and overly large parameters can force congested flows to maintain excessive backlog, making such settings suboptimal. **Third, under the same delay and application**, increasing load intensity usually shifts the optimal point toward smaller q_{min} and q_{max} . This trend is most visible for Hadoop: when the load increases from 40% to 80%, the optimal point moves toward the lower-left region under all delays. This suggests that higher load intensifies competition for shared credits, making a smaller backlog target preferable to avoid locking excessive credits in the buffer.

Although the exact optimal setting varies across network conditions, InfiniFlow is not sensitive to q_{min} and q_{max} . Across the heatmaps, most regions remain close to the optimal performance, and even in the most sensitive case, the maximum degradation is only 1.83%. Therefore, InfiniFlow does not require fine-grained parameter tuning: a broad range of parameter settings can achieve near-optimal FCT, demonstrating its robustness to parameter choices.

E Validating Prototype Fidelity

To verify the fidelity of our prototype, we conduct a validation experiment comparing the FPGA prototype against the NS-3 simulation model under identical configurations.

Experimental Setting. We test the InfiniFlow logic using a dumbbell topology. On the 100-Gbps bottleneck link, the downstream buffer is configured with a 64 KB capacity. The senders initiate two long-lived flows. The downstream drain rate is throttled to 50 Gbps. We set $q_{min} = 3$ and $q_{max} = 6$ MTUs.

Hardware-Enforced Latency Emulation. Since short-reach fiber links lack sufficient propagation delay to test BDP-related dynamics, we embedded a transparent hardware latency emulator directly into the datapath: (i) *Timestamping*: The ingress logic tags each incoming packet with a 64-bit system clock timestamp. (ii)

Lookahead and Release: Leveraging the First-Word Fall-Through (FWFT) capability of asynchronous FIFOs, the emulation logic inspects the head-of-line timestamp and asserts the read enable signal (rd_en) only after the target delay (e.g., 300 cycles for $1 \mu s$ at 300 MHz) has elapsed. This mechanism enforces precise propagation delays without compromising pipeline throughput.

Verification Results. We analyzed the timeline of four VC states, as shown in Figure 21. The hardware traces (FPGA) align closely with the simulation baselines (NS-3), exhibiting highly consistent dynamic behaviors:

Transient Response: Upon the onset of congestion ($t \approx 10 \mu s$), both platforms exhibit a sharp reduction in Credit and Remaining Buffer, coupled with a rapid spike in Inflight Packets. The Threshold responds correctly with a steep drop, effectively throttling the injection rate.

Steady-State Convergence: In contrast to unstable oscillations, the logic in both the hardware and simulation quickly converges to a stable equilibrium after $50 \mu s$. This confirms that the feedback loop effectively stabilizes the backlog within the target range.

Conclusion. The precise alignment of core control metrics confirms that our FPGA prototype serves as a faithful implementation of the InfiniFlow architecture. This validates the reliability of the large-scale simulation results presented in the main evaluation.